

GLOBAL
EDITION



Engineering Software Products

*An Introduction to Modern
Software Engineering*

Ian Sommerville



ENGINEERING SOFTWARE PRODUCTS

An Introduction to Modern Software
Engineering

Global Edition

Ian Sommerville



Pearson

Harlow, England • London • New York • Boston • San Francisco • Toronto • Sydney • Dubai • Singapore • Hong Kong
Tokyo • Seoul • Taipei • New Delhi • Cape Town • São Paulo • Mexico City • Madrid • Amsterdam • Munich • Paris • Milan

Pearson Education Limited

KAO Two
KAO Park
Hockham Way
Harlow
CM17 9SR
United Kingdom

and Associated Companies throughout the world

Visit us on the World Wide Web at: www.pearsonglobaleditions.com

© Pearson Education Limited, 2021

The rights of Ian Sommerville to be identified as the author of this work have been asserted by him in accordance with the Copyright, Designs and Patents Act 1988.

Authorized adaptation from the United States edition, entitled Engineering Software Products, 1st Edition, ISBN 978-0-13-521064-2 by Ian Sommerville, published by Pearson Education ©2020.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners. For information regarding permissions, request forms, and the appropriate contacts within the Pearson Education Global Rights and Permissions department, please visit www.pearsoned.com/permissions.

This eBook is a standalone product and may or may not include all assets that were part of the print version. It also does not provide access to other Pearson digital products like MyLab and Mastering. The publisher reserves the right to remove any material in this eBook at any time.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

ISBN 10: 1-292-37634-1

ISBN 13: 978-1-292-37634-9

eBook ISBN 13: 978-1-292-37635-6

PREFACE

Software products, such as stand-alone programs, web apps and services, and mobile apps, have transformed our everyday life and work. There are tens of thousands of software product companies, and hundreds of thousands of software engineers are employed worldwide in software product development.

Contrary to what some people may think, engineering software products needs more than coding skills. So, I've written this book to introduce some of the software engineering activities that are important for the production of reliable and secure software products.

Who is the book for?

The book has been designed for students taking a first course in software engineering. People thinking about developing a product who don't have much software engineering experience may also find it useful.

Why do we need a software engineering book that's focused on software products?

Most software engineering texts focus on *project-based* software engineering, where a client develops a specification and the software is developed by another company. However, the software engineering methods and techniques that have been developed for large-scale projects are not suited to software product development.

Students often find it difficult to relate to large, custom software systems. I think that students find it easier to understand software engineering techniques when they are relevant to the type of software that they constantly use. Also, many product engineering techniques are more directly relevant to student projects than project-oriented techniques.

Is this a new edition of your other software engineering textbook?

No, this book takes a completely different approach and, apart from a couple of diagrams, does not reuse any material from *Software Engineering*, 10th edition.

What's in the book?

Ten chapters cover software products, agile software engineering, features, scenarios and user stories, software architecture, cloud-based software, microservices architecture, security and privacy, reliable programming, testing, and DevOps and code management.

I've designed the book so that it's suitable for a one-semester software engineering course.

How is this book different from other introductory texts on software engineering?

As I said, the focus is on *products* rather than *projects*. I cover techniques that most other SE texts don't cover, such as personas and scenarios, cloud computing, microservices, security, and DevOps. As product innovation doesn't come from university research, there are no citations or references to research and the book is written in an informal style.

What do I need to know to get value from the book?

I assume that you have programming experience with a modern object-oriented programming language such as Java or Python and that you are familiar with good programming practice, such as the use of meaningful names. You should also understand basic computing concepts, such as objects, classes, and databases. The program examples in the book are written in Python, but they are understandable by anyone with programming experience.

What extra material is available to help teachers and instructors?

1. An instructor's manual with solutions to exercises and quiz questions for all chapters
2. Suggestions how you can use the book in a one-semester software engineering course
3. Presentations for teaching (Keynote, PowerPoint, and PDF)

You can access this material along with additional material at: www.pearsonglobaleditions.com

Where can I find out more?

I've written a couple of blog posts that are relevant to the book. These provide more information about my thoughts on teaching software engineering and my motivation for writing the book.

"Out with the UML (and other stuff too): reimagining introductory courses in software engineering"

<https://iansommerville.com/systems-software-and-technology/what-should-we-teach-in-software-engineering-courses/>

"Engineering Software Products"

<https://iansommerville.com/systems-software-and-technology/engineering-software-products/>

Acknowledgments

I'd like to thank the reviewers who made helpful and supportive suggestions when they reviewed the initial proposal for this book:

Paul Eggert—*UCLA Los Angeles*
Jeffrey Miller—*University of Southern California*
Harvey Siy—*University of Nebraska Omaha*
Edmund S. Yu—*Syracuse University*
Gregory Gay—*University of South Carolina*
Josh Delinger—*Towson University*
Rocky Slavin—*University of Texas San Antonio*
Bingyang Wei—*Midwestern State University*

Thanks also to Adam Barker from St. Andrews University for keeping me right on containers and to Rose Kernan who managed the production of the book.

Thanks, as ever, to my family for their help and support while I was writing the book. Particular thanks to my daughter Jane, who did a great job of reading and commenting on the text. She was a brutal editor! Her suggested changes significantly improved the quality of my prose.

Finally, special thanks to our newest family member, my beautiful grandson Cillian, who was born while I was writing this book. His bubbly personality and constant smiles were a very welcome distraction from the sometimes tedious job of book writing and editing.

Ian Sommerville

CONTENTS

Chapter 1	Software Products	11
1.1	The product vision	17
1.2	Software product management	21
1.3	Product prototyping	26
	Key Points	27
	Recommended Reading	28
	Presentations, Videos, and Links	28
	Exercises	29
Chapter 2	Agile Software Engineering	30
2.1	Agile methods	30
2.2	Extreme Programming	34
2.3	Scrum	37
	Key Points	57
	Recommended Reading	58
	Presentations, Videos, and Links	58
	Exercises	59

Chapter 3	Features, Scenarios, and Stories	60
3.1	Personas	64
3.2	Scenarios	69
3.3	User stories	76
3.4	Feature identification	80
	Key Points	89
	Recommended Reading	90
	Presentations, Videos, and Links	90
	Exercises	90
Chapter 4	Software Architecture	92
4.1	Why is architecture important?	94
4.2	Architectural design	98
4.3	System decomposition	102
4.4	Distribution architecture	113
4.5	Technology issues	119
	Key Points	123
	Recommended Reading	124
	Presentations, Videos, and Links	124
	Exercises	125
Chapter 5	Cloud-Based Software	126
5.1	Virtualization and containers	128
5.2	Everything as a service	134
5.3	Software as a service	137
5.4	Multi-tenant and multi-instance systems	142
5.5	Cloud software architecture	150
	Key Points	157
	Recommended Reading	158

Presentations, Videos, and Links	159
Exercises	159
Chapter 6 Microservices Architecture	160
6.1 Microservices	164
6.2 Microservices architecture	167
6.3 RESTful services	183
6.4 Service deployment	189
Key Points	192
Recommended Reading	193
Presentations, Videos, and Links	194
Exercises	194
Chapter 7 Security and Privacy	195
7.1 Attacks and defenses	198
7.2 Authentication	205
7.3 Authorization	211
7.4 Encryption	213
7.5 Privacy	223
Key Points	227
Recommended Reading	228
Presentations, Videos, and Links	229
Exercises	229
Chapter 8 Reliable Programming	231
8.1 Fault avoidance	233
8.2 Input validation	252
8.3 Failure management	259
Key Points	266

Recommended Reading	266
Presentations, Videos, and Links	267
Exercises	267
 Chapter 9 Testing	 269
9.1 Functional testing	272
9.2 Test automation	283
9.3 Test-driven development	291
9.4 Security testing	295
9.5 Code reviews	298
Key Points	302
Recommended Reading	302
Presentations, Videos, and Links	303
Exercises	303
 Chapter 10 DevOps and Code Management	 305
10.1 Code management	309
10.2 DevOps automation	320
10.3 DevOps measurement	331
Key Points	336
Recommended Reading	336
Presentations, Videos, and Links	337
Exercises	337
 Appendix 1	 339
Index	354

1

Software Products

This book introduces software engineering techniques that are used to develop software products. Software products are generic software systems sold to governments, businesses, and consumers. They may be designed to support a business function, such as accounting; they may be productivity tools, such as note-taking systems; or they may be games or personal information systems. Software products range in size from millions of lines of code in large-scale business systems to a few hundred lines of code in a simple app for mobile phones.

We all use software products every day on our computers, tablets, and phones. I am using a software product—the Ulysses editor—to write this book. I’ll use another editing product—Microsoft Word—to format the final version, and I’ll use Dropbox to exchange the files with the publisher. On my phone, I use software products (apps) to read email, read and send tweets, check the weather, and so on.

The engineering techniques that are used for product development have evolved from the software engineering techniques developed in the 20th century to support custom software development. When software engineering emerged as a discipline in the 1970s, virtually all professional software was “one-off,” custom software. Companies and governments wanted to automate their businesses, and they specified what they wanted their software to do. An in-house engineering team or an external software company then developed the software.

Examples of custom software that were developed around that time include:

- the U.S. Federal Aviation Administration’s air traffic management system;
- accounting systems for all of the major banks;

- billing systems for utility companies such as electricity and gas suppliers;
- military command and control systems.

Software projects were set up to develop these one-off systems, with the software system based on a set of software requirements. The contract between the software customer and the software development company included a requirements document, which was a specification of the software that should be delivered. Customers defined their requirements and worked with the development team to specify, in detail, the software's functionality and its critical attributes.

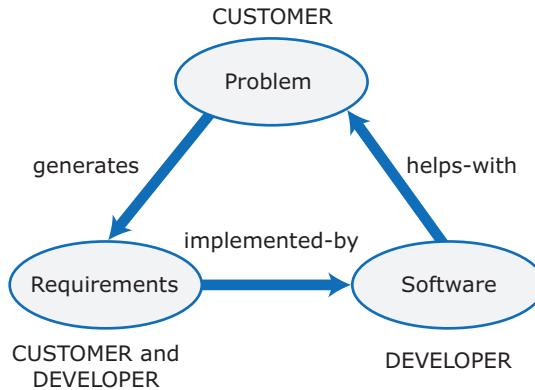
This project-based approach dominated the software industry for more than 25 years. The methods and techniques that evolved to support project-based development came to define what was meant by "software engineering." The fundamental assumption was that successful software engineering required a lot of preparatory work before starting to write programs. For example, it was important to spend time getting the requirements "right" and to draw graphical models of the software. These models were created during the software design process and used to document the software.

As more and more companies automated their business, however, it became clear that most businesses didn't really need custom software. They could use generic software products that were designed for common business problems. The software product industry developed to meet this need. Project-based software engineering techniques were adapted to software product development.

Project-based techniques are not suited to product development because of fundamental differences between project-based and product-based software engineering. These differences are illustrated in Figures 1.1 and 1.2.

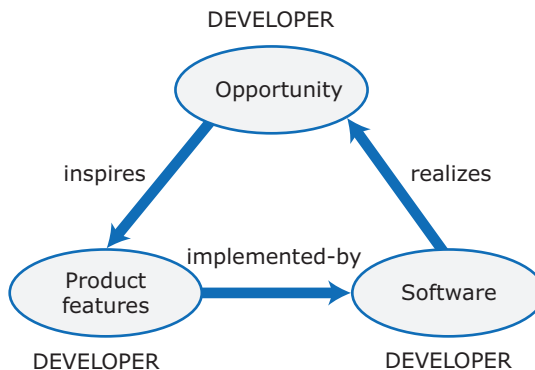
Software projects involve an external client or customer who decides on the functionality of the system and enters into a legal contract with the software development company. The customer's problem and current processes are used as a basis for creating the software requirements, which specify the software to be implemented. As the business changes, the supporting software has to change. The company using the software decides on and pays for the changes. Software often has a long lifetime, and the costs of changing large systems after delivery usually exceed the initial software development costs.

Software products are specified and developed in a different way. There is no external customer who creates requirements that define what the software

Figure 1.1 Project-based software engineering

must do. The software developer decides on the features of the product, when new releases are to be made available, the platforms on which the software will be implemented, and so on. The needs of potential customers for the software are obviously considered, but customers can't insist that the software includes particular features or attributes. The development company chooses when changes will be made to the software and when they will be released to users.

As development costs are spread over a much larger customer base, product-based software is usually cheaper, for each customer, than custom software. However, buyers of the software have to adapt their ways of working to the software, since it has not been developed with their specific needs in mind. As the developer rather than the user is in control of changes, there

Figure 1.2 Product-based software engineering

is a risk that the developer will stop supporting the software. Then the product customers will need to find an alternative product.

The starting point for product development is an opportunity that a company has identified to create a viable commercial product. This may be an original idea, such as Airbnb's idea for sharing accommodations; an improvement over existing systems, such as a cloud-based accounting system; or a generalization of a system that was developed for a specific customer, such as an asset management system.

Because the product developer is responsible for identifying the opportunity, they can decide on the features that will be included in the software product. These features are designed to appeal to potential customers so that there is a viable market for the software.

As well as the differences shown in Figures 1.1 and 1.2, there are two other important differences between project-based and product-based software engineering:

1. Product companies can decide when to change their product or take their product off the market. If a product is not selling well, the company can cut costs by stopping its development. Custom software developed in a software project usually has a long lifetime and has to be supported throughout that lifetime. The customer pays for the support and decides when and if it should end.
2. For most products, getting the product to customers quickly is critical. Excellent products often fail because an inferior product reaches the market first and customers buy that product. In practice, buyers are reluctant to change products after they have invested time and money in their initial choice.

Bringing the product to the market quickly is important for all types of products, from small-scale mobile apps to enterprise products such as Microsoft Word. This means that engineering techniques geared to rapid software development (agile methods) are universally used for product development. I explain agile methods and their role in product development in Chapter 2.

If you read about software products, you may come across two other terms: “software product lines” and “platforms” (Table 1.1). Software product lines are systems designed to be adaptable to meet the specific needs of customers by changing parts of the source code. Platforms provide a set of features that can be used to create new functionality. However, you always have to work within the constraints defined by the platform suppliers.

Table 1.1 Software product lines and platforms

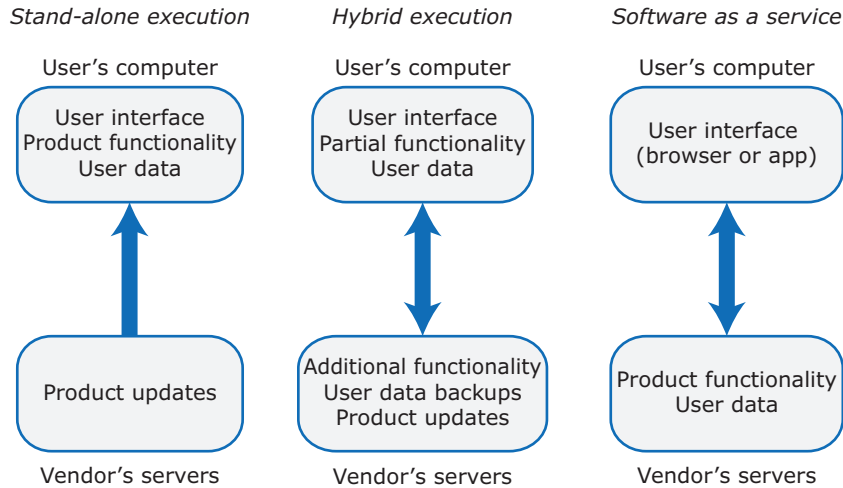
Technology	Description
Software product line	A set of software products that share a common core. Each member of the product line includes customer-specific adaptations and additions. Software product lines may be used to implement a custom system for a customer with specific needs that can't be met by a generic product. For example, a company providing communication software to the emergency services may have a software product line where the core product includes basic communication services such as receive and log calls, initiate an emergency response, pass information to vehicles, and so on. However, each customer may use different radio equipment and their vehicles may be equipped in different ways. The core product has to be adapted for each customer to work with the equipment that they use.
Platform	A software (or software+hardware) product that includes functionality so that new applications can be built on it. An example of a platform that you probably use is Facebook. It provides an extensive set of product functionality but also provides support for creating "Facebook apps." These add new features that may be used by a business or a Facebook interest group.

When software products were first developed, they were delivered on a disk and installed by customers on their computers. The software ran on those computers and user data were stored on them. There was no communication between the users' computers and the vendor's computers. Now, customers can download products from either an app store or the vendor's website.

Some products are still based on a stand-alone execution model in which all computation is carried out on the product owner's computers. However, ubiquitous high-speed networking means that alternative execution models are now available. In these models, the product owner's computers act as a client, with some or all execution and data storage on the vendor's servers (Figure 1.3).

There are two alternatives to stand-alone software products:

1. *Hybrid products* Some functionality is implemented on the user's computer and some on the product vendor's servers that are accessed over the Internet. Many phone apps are hybrid products with computationally intensive processing offloaded to remote servers.
2. *Service-based products* Applications are accessed over the Internet from a web browser or an app. There may be some local processing using

Figure 1.3 Software execution models

Javascript, but most computation is carried out on remote servers. More and more product companies are converting their products to services because it simplifies product updating and makes new business models, such as pay-as-you-go, feasible. I cover service-oriented systems in Chapters 5 and 6.

As I have said, the key characteristic of product development is that there is no external customer who generates software requirements and pays for the software. This is also true for some other types of software development:

1. *Student projects* As part of a computing or engineering course, students may be set assignments in which they work in groups to develop software. The group is responsible for deciding on the features of the system and how to work together to implement these features.
2. *Research software* Software is developed by a research team to support their work. For example, climate research depends on large-scale climate models that are designed by researchers and implemented in software. On a smaller scale, an engineering group may build software to model the characteristics of the material they are using.
3. *Internal tool development* A software development team may decide that it needs some specific tools to support their work. They specify and implement these tools as “internal” products.

You can use the product development techniques that I explain here for any type of software development that is not driven by external customer requirements.

There is a common view that software product engineering is simply advanced programming and that traditional software engineering is irrelevant. All you need to know is how to use a programming language plus the frameworks and libraries for that language. This is a misconception and I have written this book to explain the activities, apart from programming, that I believe are essential for developing high-quality software products.

If your product is to be a success, you need to think about issues other than programming. You must try to understand what your customers need and how potential users can work with your software. You need to design the overall structure of your software (software architecture) and know about technologies such as cloud computing and security engineering. You need to use professional techniques for verifying and testing your software and code management systems to keep track of a changing codebase.

You also need to think about the business case for your product. You must sell your product to survive. Creating a business case may involve market research, an analysis of competitors, and an understanding of the ways that target customers live and work. This book is about engineering, however, not business, so I don't cover business and commercial issues here.

1.1 The product vision

Your starting point for product development should be an informal “product vision.” A product vision is a simple and succinct statement that defines the essence of the product that is being developed. It explains how the product differs from other competing products. This product vision is used as a basis for developing a more detailed description of the features and attributes of the product. As new features are proposed, you should check them against the vision to make sure they contribute to it.

The product vision should answer three fundamental questions:

1. *What* is the product that you propose to develop? What makes this product different from competing products?
2. *Who* are the target users and customers for the product?
3. *Why* should customers buy this product?

The need for the first question is obvious—before you start, you need to know what you are aiming for. The other questions concern the commercial viability of the product. Most products are intended for use by customers outside of the development team. You need to understand their background to create a viable product that these customers will find attractive and be willing to buy.

If you search the web for “product vision,” you will find several variants of these questions and templates for expressing the product vision. Any of these templates can be used. The template that I like comes from the book *Crossing the Chasm* by Geoffrey Moore.¹ Moore suggests using a structured approach to writing the product vision based on keywords:

- FOR (target customer)
- WHO (statement of the need or opportunity)
- The (PRODUCT NAME) is a (product category)
- THAT (key benefit, compelling reason to buy)
- UNLIKE (primary competitive alternative)
- OUR PRODUCT (statement of primary differentiation)

On his blog *Joel on Software*, Joel Spolsky gives an example of a product described using this vision template:²

FOR a mid-sized company's marketing and sales departments WHO need basic CRM functionality, THE CRM-Innovator is a Web-based service THAT provides sales tracking, lead generation, and sales representative support features that improve customer relationships at critical touch points. UNLIKE other services or package software products, OUR product provides very capable services at a moderate cost.

You can see how this vision answers the key questions that I identified above:

1. *What* A web-based service that provides sales tracking, lead generation, and sales representative support features. The information can be used to improve relationships with customers.

¹Geoffrey Moore, *Crossing the Chasm: Marketing and selling technology products to mainstream customers* (Capstone Trade Press, 1998).

²J. Spolsky, Product Vision, 2002; <http://www.joelonsoftware.com/articles/JimHighsmithon-ProductVisi.html>

Table 1.2 Information sources for developing a product vision

Information source	Explanation
Domain experience	The product developers may work in a particular area (say, marketing and sales) and understand the software support that they need. They may be frustrated by the deficiencies in the software they use and see opportunities for an improved system.
Product experience	Users of existing software (such as word processing software) may see simpler and better ways of providing comparable functionality and propose a new system that implements this. New products can take advantage of recent technological developments such as speech interfaces.
Customer experience	The software developers may have extensive discussions with prospective customers of the product to understand the problems that they face; constraints, such as interoperability, that limit their flexibility to buy new software; and critical attributes of the software that they need.
Prototyping and “playing around”	Developers may have an idea for software but need to develop a better understanding of that idea and what might be involved in developing it into a product. They may develop a prototype system as an experiment and “play around” with ideas and variations using that prototype system as a platform.

2. *Who* The product is aimed at medium-sized companies that need standard customer relationship management software.
3. *Why* The most important product distinction is that it provides capable services at a moderate cost. It will be cheaper than alternative products.

A great deal of mythology surrounds software product visions. For successful consumer software products, the media like to present visions as if they emerge from a “Eureka moment” when the company founders have an “awesome idea” that changes the world. This view oversimplifies the effort and experimentation that are involved in refining a product idea. Product visions for successful products usually emerge after a lot of work and discussion. An initial idea is refined in stages as more information is collected and the development team discusses the practicalities of product implementation. Several different sources of information contribute to the product vision (Table 1.2).

Table 1.3 A vision statement for the iLearn system

FOR teachers and educators *WHO* need a way to help students use web-based learning resources and applications, *THE iLearn system* is an open learning environment *THAT* allows the set of resources used by classes and students to be easily configured for these students and classes by teachers themselves.

UNLIKE Virtual Learning Environments, such as Moodle, the focus of iLearn is the learning process rather than the administration and management of materials, assessments, and coursework. *OUR* product enables teachers to create subject and age-specific environments for their students using any web-based resources, such as videos, simulations, and written materials that are appropriate.

Schools and universities are the target customers for *the iLearn system* as it will significantly improve the learning experience of students at relatively low cost. It will collect and process learner analytics that will reduce the costs of progress tracking and reporting.

1.1.1 A vision example

As students, readers of this book may have used Virtual Learning Environments (VLEs), such as Blackboard and Moodle. Teachers use these VLEs to distribute class materials and assignments. Students can download the materials and upload completed assignments. Although the name suggests that VLEs are focused on learning, they are really geared to supporting learning administration rather than learning itself. They provide some features for students, but they are not open learning environments that can be tailored and adapted to a particular teacher's needs.

A few years ago, I worked on the development of a digital environment for learning support. This product was not just another VLE but was intended to provide flexible support for the process of learning. Our team looked at existing VLEs and talked to teachers and students who used them. We visited different types of school from kindergartens to colleges to examine how they used learning environments and how teachers were experimenting with software outside of these environments. We had extensive discussions with teachers about what they would like to be able to do with a digital learning environment. We finally arrived at the vision statement shown in Table 1.3.

In education, the teachers and students who use learning environments are not responsible for buying software. The purchaser is a school, university, or training center. The purchasing officer needs to know the benefits to the organization. Therefore, we added the final paragraph to the vision statement in Table 1.3 to make clear that there are benefits to organizations as well as individual learners.

1.2 Software product management

Software product management is a business activity focusing on the software products that are developed and sold by the business. Product managers (PMs) take overall responsibility for the product and are involved in planning, development, and marketing. They are the interface between the software development team, the broader organization, and the product's customers. PMs should be full members of the development team so that they can communicate business and customer requirements to the software developers.

Software product managers are involved at all stages of a product's life—from initial conception through vision development and implementation to marketing. Finally, they make decisions on when the product should be withdrawn from the market. Mid-size and large software companies may have dedicated PMs; in smaller software companies, the PM role is likely to be shared with other technical or business roles.

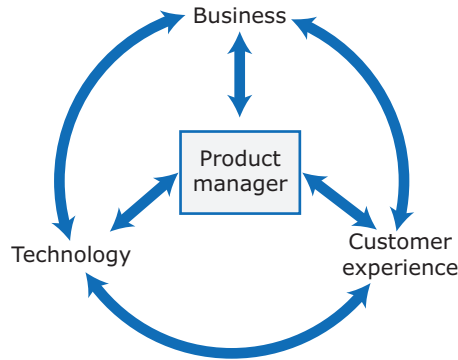
The job of the PM is to look outward to the customers and potential customers of the product rather than to focus on the software that is being developed. It is all too easy for a development team to get caught up in the details of “cool features” of the software, which most customers probably don't care about. For a product to be successful, the PM has to ensure that the development team implements features that deliver real value to customers, not just features that are technically interesting.

In a blog post, Martin Eriksson³ explains that product managers have to be concerned with business, technology, and user experience issues. Figure 1.4, which I based on Martin's diagram, illustrates these multiple concerns.

Product managers have to be generalists, with both technical and communication skills. Business, technology, and customer issues are interdependent and PMs have to consider all of them:

1. *Business needs* PMs have to ensure that the software being developed meets the business goals and objectives of both the software product company and its customers. They must communicate the concerns and needs of the customers and the development team to the managers of the product business. They work with senior managers and with marketing staff to plan a release schedule for the product.

³Based on M. Eriksson, What, exactly, is a Product Manager, 2011; <http://www.mindtheproduct.com/2011/10/what-exactly-is-a-product-manager/>

Figure 1.4 Product management concerns

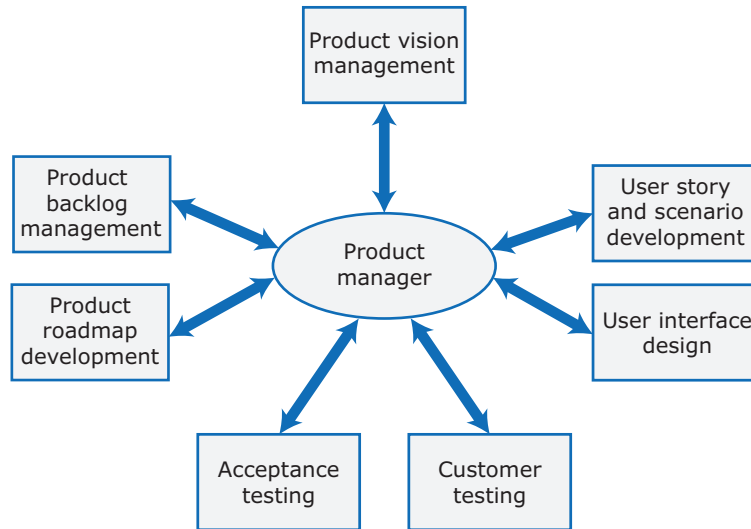
2. *Technology constraints* PMs must make developers aware of technology issues that are important to customers. These may affect the schedule, cost, and functionality of the product that is being developed.
3. *Customer experience* PMs should be in regular communication with customers to understand what they are looking for in a product, the types of user and their backgrounds, and the ways in which the product may be used. Their experience of customer capabilities is a critical input to the design of the product's user interface. PMs may also involve customers in alpha and beta product testing.

Because of the engineering focus of this book, I do not go into detail about the business role of product managers or their role in areas such as market research and financial planning. Rather, I concentrate on their interactions with the development team. PMs may interact with the development team in seven key areas (Figure 1.5).

1.2.1 Product vision management

Some writers say that the product manager should be responsible for developing the product vision. Large companies may adopt this approach, but it is often impractical in small software companies. In startups, the source of the product vision is often an original idea by the company founders. This vision is often developed long before anyone thinks about appointing a PM.

Obviously, it makes sense for PMs to take the lead in developing the product vision. They should be able to bring market and customer information to

Figure 1.5 Technical interactions of product managers

the process. However, I think all team members should be involved in vision development so that everyone can support what is finally agreed. When the team “owns” the vision, everyone is more likely to work coherently to realize that vision.

A key role of PMs is to manage the product vision. During the development process, changes are inevitably proposed by people from both inside and outside of the development team. PMs have to assess and evaluate these changes against the product vision. They must check that the changes don’t contradict the ideas embodied in the product vision. PMs also have to ensure that there is no “vision drift,” in which the vision is gradually extended to become broader and less focused.

1.2.2 Product roadmap development

A product roadmap is a plan for the development, release, and marketing of the software product. It sets out important product goals and milestones, such as the completion of critical features, the completion of the first version for user testing, and so on. It includes dates when these milestones should be reached and success criteria that help assess whether project goals have been attained. The roadmap should include a release schedule showing when

different releases of the software will be available and the key features that will be included in each release.

The development of the product roadmap should be led by the product manager but must also involve the development team as well as company managers and marketing staff. Depending on the type of product, important deadlines may have to be met if the product is to be successful. For example, many large companies must make decisions on procurement toward the end of their financial year. If you want to sell a new product to such companies, you have to make it available before then.

1.2.3 User story and scenario development

User stories and scenarios are widely used to refine a product vision to identify features of the product. They are natural language descriptions of things that users might want to do with a product. Using them, the team can decide what features need to be included and how these features should work. I cover user stories and scenarios in Chapter 3.

The product manager's job is to understand the product's customers and potential customers. PMs should therefore lead the development of user scenarios and stories, which should be based on knowledge of the area and of the customer's business. PMs should also take scenarios and stories suggested by other team members back to customers to check that they reflect what the target users of the product might actually do.

1.2.4 Product backlog management

In product development, it is important for the process to be driven by a “product backlog.” A product backlog is a to-do list that sets out what has to be done to complete the product development. The backlog is added to and refined incrementally during the development process. I explain how product backlogs are used in the Scrum method in Chapter 2.

The product manager plays a critical role as the authority on the product backlog items that should take priority for development. PMs also help to refine broad backlog items, such as “implement auto-save,” in more detail at each project iteration. If suggestions for change are made, it is up to the PM to decide whether or not the product backlog should be rearranged to prioritize the suggested changes.

1.2.5 Acceptance testing

Acceptance testing is the process of verifying that a software release meets the goals set out in the product roadmap and that the product is efficient and reliable. Product managers should be involved in developing tests of the product features that reflect how customers use the product. They may work through usage scenarios to check that the product is ready to be released to customers.

Acceptance tests are refined as the product is developed, and products must pass these tests before being released to customers.

1.2.6 Customer testing

Customer testing involves taking a release of a product to existing and potential customers and getting feedback from them on the product's features, its usability, and the fit of the product to their business. Product managers are involved in selecting customers that might be interested in taking part in the customer testing process and working with them during that process. They have to ensure that the customer can use the product and that the customer testing process collects useful information for the development team.

1.2.7 User interface design

The user interface (UI) of a product is critical in the commercial acceptance of a software product. Technically excellent products are unlikely to be commercially successful if users find them difficult to use or if their UI is incompatible with other software that they use. UI design is challenging for small development teams because most users are less technically skilled than software developers. It is often difficult for developers to envision the problems that users may have with a software product.

Product managers should understand user limitations and act as surrogate users in their interactions with the development team. They should evaluate UI features as they are developed to check that these features are not unnecessarily complex or force users to work in an unnatural way. PMs may arrange for potential users to try out the software, comment on its UI, and assist with designing error messages and a help system.

1.3 Product prototyping

Product prototyping is the process of developing an early version of a product to test your ideas and to convince yourself and company funders that your product has real market potential. You use a product prototype to check that what you want to do is feasible and to demonstrate your software to potential customers and funders. Prototypes may also help you understand how to organize and structure the final version of your product.

You may be able to write an inspiring product vision, but your potential users can only really relate to your product when they see a working version of your software. They can point out what they like and don't like about it and make suggestions for new features. Venture capitalists, whom you may approach for funding, usually insist on seeing a product prototype before they commit to supporting a startup company. The prototype plays a critical role in convincing investors that your product has commercial potential.

A prototype may also help identify fundamental software components or services and test technology. You may find that the technology you planned to use is inadequate and that you have to revise your ideas on how to implement the software. For example, you may discover that the design you chose for the prototype cannot handle the expected load on the system, so you have to redesign the overall product architecture.

Building a prototype should be the first thing you do when developing a software product. Your goal should be to have a working version of your software that can be used to demonstrate its key features. A short development cycle is critical; you should aim to have a demonstrable system up and running in four to six weeks. Of course, you have to cut corners to do this, so you may choose to ignore issues such as reliability and performance and work with a rudimentary user interface.

Sometimes prototyping is a two-stage process:

1. *Feasibility demonstration* You create an executable system that demonstrates the new ideas in your product. The goals at this stage are to see whether your ideas actually work and to show funders and company management that your product features are better than those of competitors.
2. *Customer demonstration* You take an existing prototype created to demonstrate feasibility and extend it with your ideas for specific customer features and how these can be realized. Before you develop a customer prototype,

you need to do some user studies and have a clear idea of your potential users and scenarios of use. I explain how to develop user personas and usage scenarios in Chapter 3.

You should always use technology that you know and understand to develop a prototype so that you don't have to spend time learning a new language or framework. You don't need to design a robust software architecture. You may leave out security features and checking code to ensure software reliability. However, I recommend that, for prototypes, you should always use automated testing and code management. These are covered in Chapters 9 and 10.

If you are developing software without an external customer, such as software for a research group, it may be that a prototype system is all you need. You can develop and refine the prototype as your understanding of the problem develops. However, as soon as you have external users of your software, you should always think of your prototype as a “throw-away” system. The inevitable compromises and shortcuts you make to speed up development result in prototypes that become increasingly difficult to change and evolve to include new features. Adding security and reliability may be practically impossible.

KEY POINTS

- Software products are software systems that include general functionality that is likely to be useful to a wide range of customers.
- In product-based software engineering, the same company is responsible for deciding on both the features that should be part of the product and the implementation of these features.
- Software products may be delivered as stand-alone products running on the customer's computers, hybrid products, or service-based products. In hybrid products, some features are implemented locally and others are accessed from the Internet. All features are remotely accessed in service-based products.
- A product vision succinctly describes what is to be developed, who are the target customers for the product, and why customers should buy the product you are developing.
- Domain experience, product experience, customer experience, and an experimental software prototype may all contribute to the development of the product vision.

- Key responsibilities of product managers are to own the product vision, develop a product roadmap, create user stories and scenarios, manage the product backlog, conduct customer and acceptance testing, and design the user interface.
- Product managers work at the interface between the business, the software development team, and the product customers. They facilitate communication among these groups.
- You should always develop a product prototype to refine your own ideas and to demonstrate the planned product features to potential customers.

RECOMMENDED READING

“What is Product Line Engineering?” This article and the two linked articles provide an overview of software product line engineering and highlight the differences between product line engineering and software product development. (Biglever Software, 2013)

<http://www.productlineengineering.com/overview/what-is-ple.html>

“Building Software Products vs Platforms” This blog post briefly explains the differences between a software product and a software platform. (B. Algave, 2016)

<https://blog.frogslayer.com/building-software-products-vs-platforms/>

“Product Vision” This is an old article but an excellent summary of what is meant by a product vision and why it is important. (J. Spolsky, 2002)

<http://www.joelonsoftware.com/articles/JimHighsmithonProductVisi.html>

Agile Product Management with Scrum I generally avoid recommending books on product management as they are too detailed for most readers of this book. However, this book is worth looking at because of its focus on software and its integration with the Scrum agile method that I cover in Chapter 2. It’s a short book that includes a succinct introduction to product management and discusses the creation of a product vision. (R. Pichler, 2010, Addison-Wesley)

The author’s blog also has articles on product management.

<http://www.romanpichler.com/blog/romans-product-management-framework/>

“What, Exactly, is a Product Manager?” This excellent blog post explains why it’s important that product managers work at the intersection of business, technology, and users. (M. Eriksson, 2011)

<http://www.mindtheproduct.com/2011/10/what-exactly-is-a-product-manager/>

PRESENTATIONS, VIDEOS, AND LINKS

<https://iansommerville.com/engineering-software-products/software-products>

EXERCISES

- 1.1. Briefly describe the fundamental differences between project-based and product-based software engineering.
- 1.2. What are three important differences between software products and software product lines.
- 1.3. Based on the example project vision for the iLearn system, identify the WHAT, WHO, and WHY for that software product.
- 1.4. Why do software product managers have to be generalists, with a range of skills, rather than simply technical specialists?
- 1.5. You are a software product manager for a company developing educational software products based on scientific simulations. Explain why it is important to develop a product roadmap so that final product releases are available in the first three months of the year.
- 1.6. Why should you implement a prototype before you start developing a new software product?

2

Agile Software Engineering

Bringing a software product to the market quickly is critically important. This is true for all types of products—from simple mobile apps to large-scale enterprise products. If a product is released later than planned, a competitor may have already captured the market or you may have missed a market window, such as the beginning of the holiday season. Once users have committed to a product, they are usually reluctant to change, even to a technically superior product.

Agile software engineering focuses on delivering functionality quickly, responding to changing product specifications, and minimizing development overheads. An “overhead” is any activity that doesn’t contribute directly to rapid product delivery. Rapid development and delivery and the flexibility to make changes quickly are fundamental requirements for product development.

A large number of “agile methods” have been developed. Each has its adherents, who are often evangelical about the method’s benefits. In practice, companies and individual development teams pick and choose agile techniques that work for them and that are most appropriate for their size and the type of product they are developing. There is no best agile method or technique. It depends on who is using the technique, the development team, and the type of product being developed.

2.1 Agile methods

In the 1980s and early 1990s, there was a widespread view that the best way to create good software was to use controlled and rigorous software development processes. The processes included detailed project planning, requirements

specification and analysis, the use of analysis and design methods supported by software tools, and formal quality assurance. This view came from the software engineering community that was responsible for developing large, long-lived software systems such as aerospace and government systems. These were “one-off” systems, based on the customer requirements.

This approach is sometimes called plan-driven development. It evolved to support software engineering where large teams developed complex, long-lifetime systems. Teams were often geographically dispersed and worked on the software for long periods of time. An example of this type of software is a control system for a modern aircraft. Developing an avionic system might take five to ten years from initial specification to on-board deployment.

Plan-driven development involves significant overhead in planning, designing, and documenting the system. This overhead is justifiable for critical systems where the work of several development teams must be coordinated and different people may maintain and update the software during its lifetime. Detailed documents describing the software requirements and design are important when informal team communications are impossible.

If plan-driven development is used for small and medium-sized software products, however, the overhead involved is so large that it dominates the software development process. Too much time is spent writing documents that may never be read rather than writing code. The system is specified in detail before implementation begins. Specification errors, omissions, and misunderstandings are often discovered only after a significant chunk of the system has been implemented.

To fix these problems, developers have to redo work that they thought was complete. As a consequence, it is practically impossible to deliver software quickly and to respond rapidly to requests for changes to the delivered software.

Dissatisfaction with plan-driven software development led to the creation of agile methods in the 1990s. These methods allowed the development team to focus on the software itself, rather than on its design and documentation. Agile methods deliver working software quickly to customers, who can then propose new or different requirements for inclusion in later versions of the system. They reduce process bureaucracy by avoiding work that has dubious long-term value and eliminating documentation that will probably never be used.

The philosophy behind agile methods is reflected in the agile manifesto¹ that was agreed on by the leading developers of these methods. Table 2.1 shows the key message in the agile manifesto.

¹Retrieved from <http://agilemanifesto.org/>. Used with permission.

Table 2.1 The agile manifesto

We are uncovering better ways of developing software by doing it and helping others to do it. Through this work, we have come to value:

- individuals and interactions over processes and tools;
- working software over comprehensive documentation;
- customer collaboration over contract negotiation;
- responding to change over following a plan.

While there is value on the items on the right, we value the items on the left more.

All agile methods are based on incremental development and delivery. The best way to understand incremental development is to think of a software product as a set of features. Each feature does something for the software user. There might be a feature that allows data to be entered, a feature to search the entered data, and a feature to format and display the data. Each software increment should implement a small number of product features.

With incremental development, you delay decisions until you really need to make them. You start by prioritizing the features so that the most important features are implemented first. You don't worry about the details of all the features—you define only the details of the feature that you plan to include in an increment. That feature is then implemented and delivered. Users or surrogate users can try it out and provide feedback to the development team. You then go on to define and implement the next feature of the system.

I show this process in Figure 2.1, and I describe incremental development activities in Table 2.2.

Figure 2.1 Incremental development

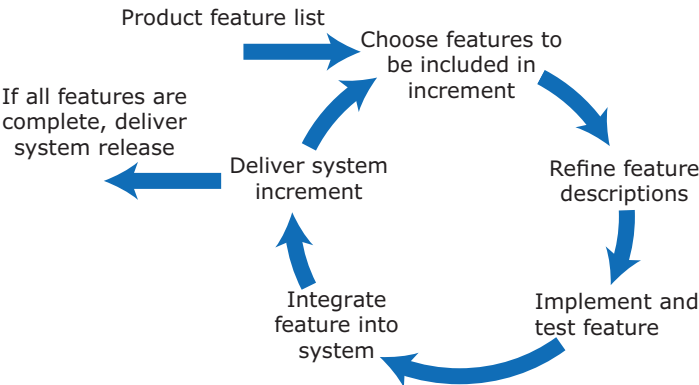


Table 2.2 Incremental development activities

Activity	Description
Choose features to be included in an increment	Using the list of features in the planned product, select those features that can be implemented in the next product increment.
Refine feature descriptions	Add detail to the feature descriptions so that the team members have a common understanding of each feature and there is sufficient detail to begin implementation.
Implement and test	Implement the feature and develop automated tests for that feature that show that its behavior is consistent with its description. I explain automated testing in Chapter 9.
Integrate feature and test	Integrate the developed feature with the existing system and test it to check that it works in conjunction with other features.
Deliver system increment	Deliver the system increment to the customer or product manager for checking and comments. If enough features have been implemented, release a version of the system for customer use.

Of course, reality doesn't always match this simple model of feature development. Sometimes an increment has to be devoted to developing an infrastructure service, such as a database service, that is used by several features; sometimes you need to plan the user interface so that you get a consistent interface across features; and sometimes an increment has to sort out problems, such as performance issues, that were discovered during system testing.

All agile methods share a set of principles based on the agile manifesto, so they have much in common. I summarize these agile principles in Table 2.3.

Almost all software products are now developed with an agile approach. Agile methods work for product engineering because software products are usually stand-alone systems rather than systems composed of independent subsystems. They are developed by co-located teams who can communicate informally. The product manager can easily interact with the development team. Consequently, there is no need for formal documents, meetings, and cross-team communication.

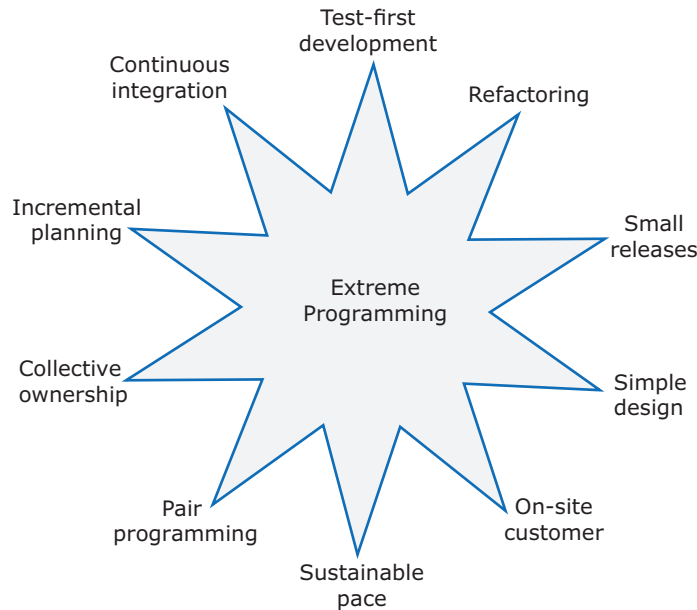
Table 2.3 Agile development principles

Principle	Description
Involve the customer	Involve customers closely with the software development team. Their role is to provide and prioritize new system requirements and to evaluate each increment of the system.
Embrace change	Expect the features of the product and the details of these features to change as the development team and the product manager learn more about the product. Adapt the software to cope with changes as they are made.
Develop and deliver incrementally	Always develop software products in increments. Test and evaluate each increment as it is developed and feed back required changes to the development team.
Maintain simplicity	Focus on simplicity in both the software being developed and the development process. Wherever possible, do what you can to eliminate complexity from the system.
Focus on people, not the development process	Trust the development team and do not expect everyone to always do things in the same way. Team members should be left to develop their own ways of working without being limited by prescriptive software processes.

2.2 Extreme Programming

The ideas underlying agile methods were developed by a number of different people in the 1990s. However, the most influential work that has changed the culture of software development was the development of Extreme Programming (XP). The name was coined by Kent Beck in 1998 because the approach pushed recognized good practice, such as iterative development, to “extreme” levels. For example, regular integration, in which the work of all programmers in a team is integrated and tested, is good software engineering practice. XP advocates that changed software should be integrated several times per day, as soon as the changes have been tested.

XP focused on new development techniques that were geared to rapid, incremental software development, change, and delivery. Figure 2.2 shows 10 fundamental practices, proposed by the developers of Extreme Programming, that characterize XP.

Figure 2.2 Extreme Programming practices

The developers of XP claim that it is a holistic approach. All of these practices are essential. In reality, however, development teams pick and choose the techniques that they find useful given their organizational culture and the type of software they are writing. Table 2.4 describes XP practices that have become part of mainstream software engineering, particularly for software product development. The other XP practices shown in Figure 2.2 have been less widely adopted but are used in some companies.

I cover these widely-used XP practices, in later chapters of the book. Incremental planning and user stories are covered in Chapter 3, refactoring in Chapter 8, test-driven development in Chapter 9, and continuous integration and small releases in Chapter 10.

You may be surprised that “Simple design” is not on the list of popular XP practices. The developers of XP suggested that the “YAGNI” (You Ain’t Gonna Need It) principle should apply when designing software. You should include only functionality that is requested, and you should not add extra code to cope with situations anticipated by the developers. This sounds like a great idea.

Unfortunately, it ignores the fact that customers rarely understand system-wide issues such as security and reliability. You need to design and implement software to take these issues into account. This usually means including code to cope with situations that customers are unlikely to foresee and describe in user stories.

Table 2.4 Widely adopted XP practices

Practice	Description
Incremental planning/ user stories	There is no “grand plan” for the system. Instead, what needs to be implemented (the requirements) in each increment are established in discussions with a customer representative. The requirements are written as user stories. The stories to be included in a release are determined by the time available and their relative priority.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the previous release.
Test-driven development	Instead of writing code and then tests for that code, developers write the tests first. This helps clarify what the code should actually do and that there is always a “tested” version of the code available. An automated unit test framework is used to run the tests after every change. New code should not “break” code that has already been implemented.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system and a new version of the system is created. All unit tests from all developers are run automatically and must be successful before the new version of the system is accepted.
Refactoring	Refactoring means improving the structure, readability, efficiency, and security of a program. All developers are expected to refactor the code as soon as potential code improvements are found. This keeps the code simple and maintainable.

Practices such as having an on-site customer and collective ownership of code are good ideas. An on-site customer works with the team, proposes stories and tests, and learns about the product. However, the reality is that customers and surrogate customers such as product managers have many other things to do. It is difficult for them to find the time to be fully embedded in a development team.

Collective ownership discourages the individual ownership of code, but it has proved to be impractical in many companies. Specialists are needed for some types of code. Some people may work part-time on a project and so cannot participate in its “ownership.” Some team members may be psychologically unsuited to this way of working and have no wish to “own” someone else’s code.

In pair programming two developers create each code unit. It was proposed by the inventors of XP because they believed the pair could learn from each other and catch each other's mistakes. They suggested that two people working together were more productive than two people working as individuals. However, there is no hard evidence that pair programming is more productive than individual work. Many managers consider pair programming to be unproductive because two people seem to be doing one job.

Working at a sustainable pace, with no overtime, is attractive in principle. Team members should be more productive if they are not tired and stressed. However, it is difficult to convince managers that this sustainable working will help meet tight delivery deadlines.

Extreme programming considers management to be a collective team activity; normally, there is no designated project manager responsible for communicating with management and planning the work of the team. In fact, software development is a business activity and so has to fit with broader business concerns of financing, costs, schedules, hiring and managing staff, and maintaining good customer relationships. This means that management issues cannot simply be left to the development team. There needs to be explicit management where a manager can take account of business needs and priorities as well as technical issues.

2.3 Scrum

In any software business, managers need to know what is going on and whether or not a software development project is likely to deliver the software on time and within its budget. Traditionally, this involves drawing up a project plan that shows a set of milestones (what will be achieved), deliverables (what will be delivered by the team), and deadlines (when a milestone will be reached). The “grand plan” for the project shows everything from start to finish. Progress is assessed by comparing that plan with what has been achieved.

The problem with up-front project planning is that it involves making detailed decisions about the software long before implementation begins. Inevitably things change. New requirements emerge, team members come and go, business priorities evolve, and so on. Almost from the day they are formulated, project plans have to change. Sometimes this means that “finished” work has to be redone. This is inefficient and often delays the final delivery of the software.

On this basis, the developers of agile methods argued that plan-based management is wasteful and unnecessary. It is better to plan incrementally so that the plan can change in response to changing circumstances. At the start of each development cycle, decisions are made on what features should be prioritized, how these should be developed and what each team member should do. Planning should be informal with minimal documentation and with no designated project manager.

Unfortunately, this informal approach to management does not meet the broader business need of progress tracking and assessment. Senior managers do not have the time to become involved in detailed discussions with team members. Managers want someone who can report on progress and take their concerns and priorities back to the development team. They need to know whether the software will be ready by the planned completion date, and they need information to update their business plan for the product.

This requirement for a more proactive approach to agile project management led to the development of Scrum. Unlike XP, Scrum is not based on a set of technical practices. Rather, it is designed to provide a framework for agile project organization with designated individuals (the ScrumMaster and the Product Owner) who act as the interface between the development team and the organization.

The developers of Scrum wanted to emphasize that these individuals were not “traditional” project managers who have the authority to direct the team. So they invented new Scrum terminology for both individuals and team activities (Table 2.5). You need to know this Scrum jargon to understand the Scrum method.

Two key roles in Scrum are not part of other methods:

1. The *Product Owner* is responsible for ensuring that the development team always focuses on the product they are building rather than diverted to technically interesting but less relevant work. In product development, the product manager should normally take on the Product Owner role.
2. The *ScrumMaster* is a Scrum expert whose job is to guide the team in the effective use of the Scrum method. The developers of Scrum emphasize that the ScrumMaster is not a conventional project manager but is a coach for the team. The ScrumMaster has the authority within the team on how Scrum is used. However, in many companies that use Scrum, the ScrumMaster also has some project management responsibilities.

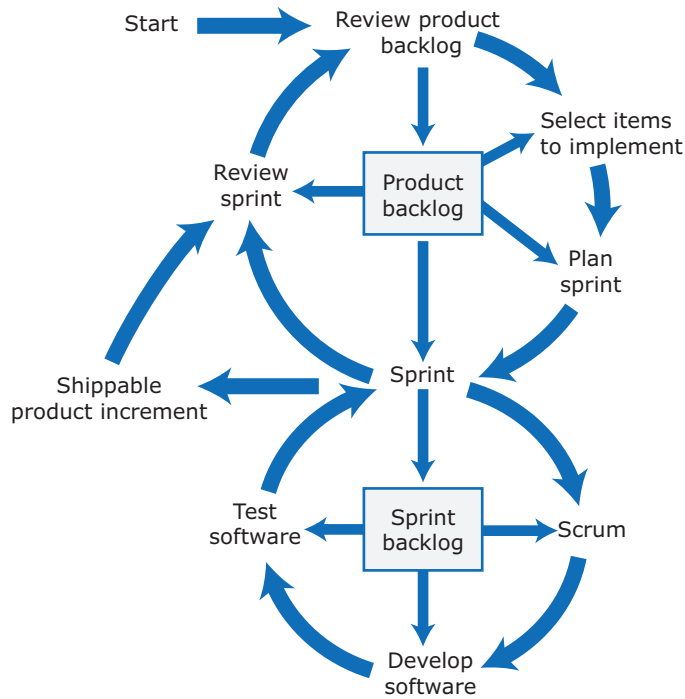
Table 2.5 Scrum terminology

Scrum term	Explanation
Product	The software product that is being developed by the Scrum team.
Product Owner	A team member who is responsible for identifying product features and attributes. The Product Owner reviews work done and helps to test the product.
Product backlog	A to-do list of items such as bugs, features, and product improvements that the Scrum team has not yet completed.
Development team	A small self-organizing team of five to eight people who are responsible for developing the product.
Sprint	A short period, typically two to four weeks, when a product increment is developed.
Scrum	A daily team meeting where progress is reviewed and work to be done that day is discussed and agreed.
ScrumMaster	A team coach who guides the team in the effective use of Scrum.
Potentially shippable product increment	The output of a sprint that is of high enough quality to be deployed for customer use.
Velocity	An estimate of how much work a team can do in a single sprint.

The other Scrum term that may need explanation is “potentially shippable product increment.” This means that the outcome of each sprint should be product-quality code. It should be completely tested, documented, and, if necessary, reviewed. Tests should be delivered with the code. There should always be a high-quality system available that can be demonstrated to management or potential customers.

The Scrum process or sprint cycle is shown in Figure 2.3. The fundamental idea underlying the Scrum process is that software should be developed in a series of “sprints.” A sprint is a fixed-length (timeboxed) activity, with each sprint normally lasting two to four weeks. During a sprint, the team has daily meetings (Scrums) to review the work done so far and to agree on that day’s activities. The “sprint backlog” is used to keep track of work that is to be done during that sprint.

Sprint planning is based on the product backlog, which is a list of all the activities that have to be completed to finish the product being developed. Before a new sprint starts, the product backlog is reviewed. The highest-priority

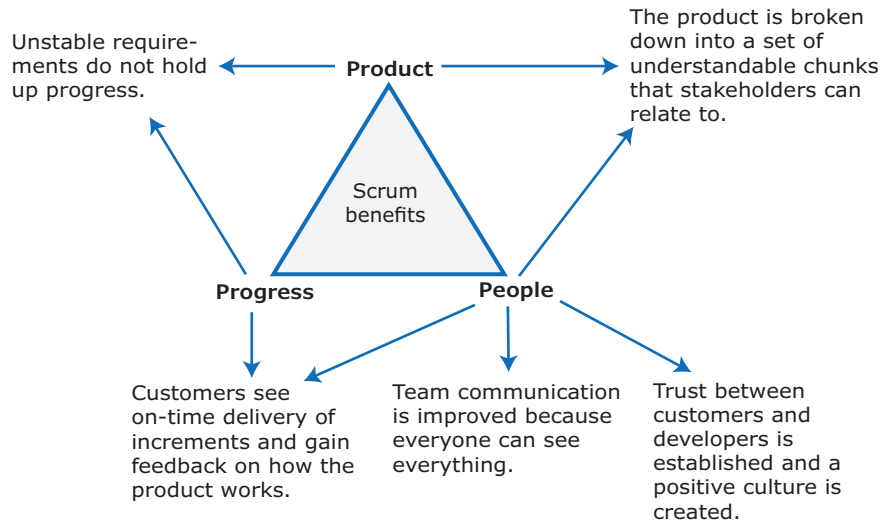
Figure 2.3 Scrum cycle

items are selected for implementation in the next sprint. Team members work together to plan the sprint by analyzing the selected items to create the sprint backlog. This is a list of activities to be completed during the sprint.

During implementation, the team implements as many of the sprint backlog items as they can in the fixed time period allowed for the sprint. Incomplete items are returned to the product backlog. Sprints are never extended to finish an incomplete item.

A sprint produces either a shippable product increment that can be delivered to customers or an internal deliverable. Internal deliverables, such as a product prototype or an architectural design, provide information for future sprints. If the sprint output is part of the final product, it should be complete. Unless the team has to change the software functionality, it should not have to do any more work on that software increment in future sprints.

On completion of a sprint, a review meeting is held involving all team members. The team discusses what went well during the sprint, what problems arose, and how the problems were tackled. Team members also reflect on the effectiveness of the tools and methods used. The aim of this meeting is for the team to learn from each other to avoid problems and to improve productivity in later sprints.

Figure 2.4 The top five benefits of using Scrum

The key benefits that come from using Scrum relate to the product being developed, the progress of the project, and the people involved (Figure 2.4).

Scrum has been very influential in the development of agile software engineering. It provides a framework for “doing” software engineering without prescribing the engineering techniques that should be used. However, Scrum is prescriptive in defining roles and the Scrum process. In *The Scrum Guide*², the “keepers” of the Scrum method state:

Scrum’s roles, artefacts, events, and rules are immutable and although implementing only parts of Scrum is possible, the result is not Scrum. Scrum exists only in its entirety and functions well as a container for other techniques, methodologies, and practices.

That is, they believe you should not pick and choose a subset of Scrum practices. Rather, you should take the whole of the method on board. It seems to me that this inflexibility contradicts the fundamental agile principle that individuals and interactions should be preferred over processes and tools. This principle suggests that individuals should be able to adapt and modify Scrum to suit their circumstances.

In some circumstances, I think it makes sense to use some of the ideas from Scrum without strictly following the method or defining the roles as exactly envisaged in Scrum. In general, “pure Scrum” with its various roles can’t

²*The Scrum Guide* This definitive guide to the Scrum method defines all the Scrum roles and activities. (K. Schwaber and J. Sutherland, 2013).

Table 2.6 Examples of product backlog items

1. As a teacher, I want to be able to configure the group of tools that are available to individual classes. (feature)
2. As a parent, I want to be able to view my children’s work and the assessments made by their teachers. (feature)
3. As a teacher of young children, I want a pictorial interface for children with limited reading ability. (user request)
4. Establish criteria for the assessment of open source software that might be used as a basis for parts of this system. (development activity)
5. Refactor user interface code to improve understandability and performance. (engineering improvement)
6. Implement encryption for all personal user data. (engineering improvement)

be used by teams with fewer than five people. So, if you are working with a smaller development team, you have to modify the method.

Small software development teams are the norm in startups, where the whole company may be the development team. They are also common in educational and research settings, where teams develop software as part of their learning, and in larger manufacturing companies, where software development is part of a broader product development process.

I think that motivated teams should make their own decisions about how to use Scrum or a Scrum-like process. However, I recommend that three important features of Scrum should be part of any product development process: product backlogs, timeboxed sprints, and self-organizing teams.

2.3.1 Product backlogs

The product backlog is a list of what needs to be done to complete the development of the product. The items on this list are called product backlog items (PBIs). The product backlog may include a variety of different items such as product features to be implemented, user requests, essential development activities, and desirable engineering improvements. The product backlog should always be prioritized so that the items that will be implemented first are at the top of the list.

Product backlog items are initially described in broad terms without much detail. For example, the items shown in Table 2.6 might be included in the product backlog for a version of the iLearn system, which I introduced in Chapter 1. In Chapter 3 I explain how system features can be identified from a product vision. These then become PBIs. I also explain how user stories can be used to identify PBIs.

Table 2.7 Product backlog item states

Heading	Description
Ready for consideration	These are high-level ideas and feature descriptions that will be considered for inclusion in the product. They are tentative so may radically change or may not be included in the final product.
Ready for refinement	The team has agreed that this is an important item that should be implemented as part of the current development. There is a reasonably clear definition of what is required. However, work is needed to understand and refine the item.
Ready for implementation	The PBI has enough detail for the team to estimate the effort involved and to implement the item. Dependencies on other items have been identified.

Table 2.6 shows different types of product backlog items. The first three items are user stories that are related to features of the product that have to be implemented. The fourth item is a team activity. The team must spend time deciding how to select open-source software that may be used in later increments. This type of activity should be specifically accounted for as a PBI rather than taken as an implicit activity that takes up team members' time. The last two items are concerned with engineering improvements to the software. These don't lead to new software functionality.

PBIs may be specified at a high level and the team decides how to implement these items. For example, the development team is best placed to decide how to refactor code for efficiency and understandability. It does not make sense to refine this item in more detail at the start of a sprint. However, high-level feature definitions usually need refinement so that team members have a clear idea of what is required and can estimate the work involved.

Items in the product backlog are considered to be in one of three states, as shown in Table 2.7. The product backlog is continually changed and extended during the project as new items are added and items in one state are analyzed and moved to a more refined state.

A critical part of the Scrum agile process is the product backlog review, which should be the first item in the sprint planning process. In this review, the product backlog is analyzed and backlog items are prioritized and refined. Backlog reviews may also take place during sprints as the team learns more about the system. Team members may modify or refine existing backlog items or add new items to be implemented in a later sprint. During a product backlog review, items may be moved from one state to another.

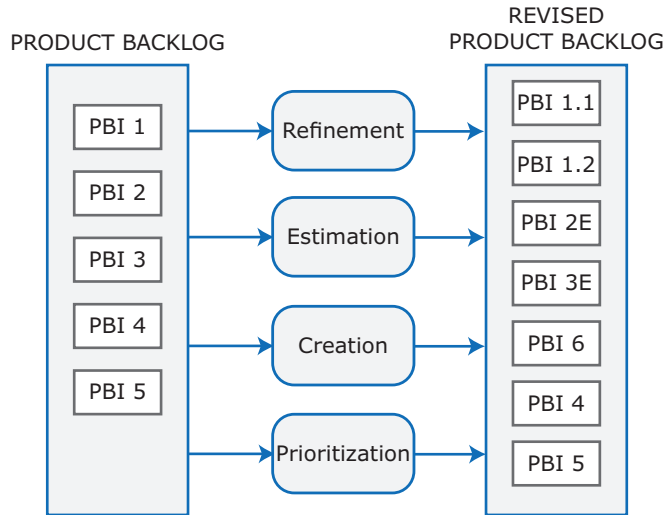
Figure 2.5 Product backlog activities

Figure 2.5 shows the four operations that may modify the product backlog. In this example, backlog item 1 has been split into two items, items 2 and 3 have been estimated, items 4 and 5 have been re-prioritized, and item 6 has been added. Notice that the new item 6 has a higher priority than existing items 4 and 5.

The Scrum community sometimes uses the term “backlog grooming” to cover these four activities:

1. *Refinement* Existing PBIs are analyzed and refined to create more detailed PBIs. This may also lead to the creation of new backlog items.
2. *Estimation* The team estimates the amount of work required to implement a PBI and adds this assessment to each analyzed PBI.
3. *Creation* New items are added to the backlog. These may be new features suggested by the product manager, required feature changes, engineering improvements, or process activities such as the assessment of development tools that might be used.
4. *Prioritization* The PBIs are reordered to take new information and changed circumstances into account.

Backlog prioritization is a whole-team activity in which decisions are made on which items to work on during a sprint. Input from product managers is essential because they should know about customer needs and priorities. The

highest-priority items are refined by the development team to create a “sprint backlog,” which is a list of more detailed implementation items. In situations such as speculative product development or research system development, where there is no specified product owner, the team should collectively prioritize the items.

Items that are ready for implementation should always have an associated estimate of how much effort is needed to implement them. Estimates are essential for sprint planning because a team uses them to decide how much work they can take on for an individual sprint. This effort estimate is an input to the prioritization activity. Sometimes it makes sense to place a higher priority on the items that deliver the most value for the least effort.

PBI estimates provide an indication of the effort required to complete each item. Two metrics are commonly used:

1. *Effort required* The amount of effort may be expressed in person-hours or person-days—that is, the number of hours or days it would take one person to implement that PBI. This is not the same as calendar time. Several people may work on an item, which may shorten the calendar time required. Alternatively, a developer may have other responsibilities that prevent full-time work on a project. Then the calendar time required is longer than the effort estimate.
2. *Story points* Story points are an arbitrary estimate of the effort involved in implementing a PBI, taking into account the size of the task, its complexity, the technology that may be required, and the “unknown” characteristics of the work. Story points were derived originally by comparing user stories, but they can be used for estimating any kind of PBI. Story points are estimated relatively. The team agrees on the story points for a baseline task. Other tasks are then estimated by comparison with this baseline—for example, more or less complex, larger or smaller, and so on. The advantage of story points is that they are more abstract than effort required because all story points should be the same, irrespective of individual abilities.

Effort estimation is hard, especially at the beginning of a project when a team has little or no previous experience with this type of work or when technologies new to the team are used. Estimates are based on the subjective judgment of the team members, and initial estimates are inevitably wrong. Estimates usually improve, however, as the team gains experience with the product and its development process.

The Scrum method recommends a team-based estimation approach called “Planning Poker,” which I don’t go into here. The rationale is that teams should be able to make better estimates than individuals. However, there is no convincing empirical evidence showing that collective estimation is better than estimates made by experienced, individual developers.

After a number of sprints have been completed, it becomes possible for a team to estimate its “velocity.” Simplistically, a team’s velocity is the sum of the size estimates of the items that have been completed during a fixed-time sprint. For example, assume that PBIs are estimated in story points and, in consecutive sprints, the team implements 17, 14, 16, and 19 story points. The team’s velocity is therefore between 16 and 17 story points per sprint.

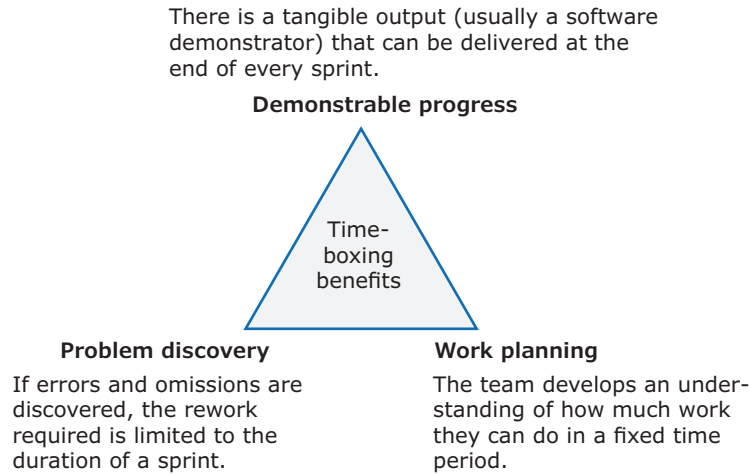
Velocity is used to decide how many PBIs a team can realistically commit to in each sprint. In the above example, the team should commit to about 17 story points. Velocity may also be used as a measure of productivity. Teams should try to refine how they work so that their velocity improves over the course of a project.

The product backlog is a shared, “living” document that is regularly updated during product development. It is usually too large to fit on a whiteboard, so it makes sense to maintain it as a shared digital document. Several specialized tools that support Scrum include facilities to share and revise product backlogs. Some companies may decide to buy these tools for their software developers.

Small companies or groups with limited resources can use a shared document system such as Office 365 or Google docs. These low-cost systems don’t require new software to be bought and installed. If you are starting out using product backlogs in your development process, I recommend this general approach to gain experience before you decide whether you need specialized tools for backlog management.

2.3.2 Timeboxed sprints

A Scrum concept that is useful in any agile development process is timeboxed sprints. Timeboxing means that a fixed time is allocated for completing an activity. At the end of the timebox, work on the activity stops whether or not the planned work has been completed. Sprints are short activities (one to four weeks) and take place between defined start and end dates. During a sprint, the team works on the items from the product backlog. The product is therefore developed in a series of sprints, each of which delivers an increment of the product or supporting software.

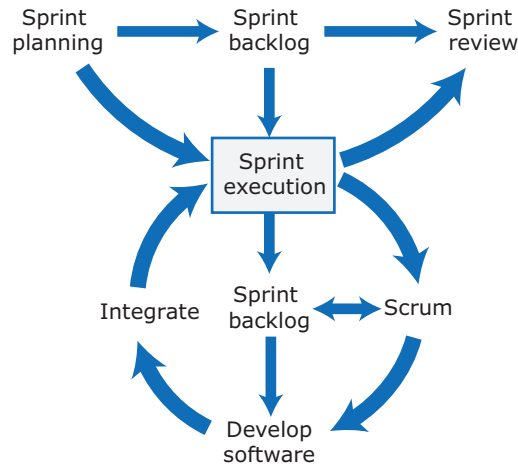
Figure 2.6 Benefits of using timeboxed sprints

Incremental development is a fundamental part of all agile methods, and I think Scrum has got it right in insisting that the time spent on each increment should be the same. In Figure 2.6, I show three important benefits that arise from using timeboxed sprints.

Every sprint involves three fundamental activities:

1. *Sprint planning* Work items to be completed during that sprint are selected and, if necessary, refined to create a sprint backlog. This should not last more than a day at the beginning of the sprint.
2. *Sprint execution* The team works to implement the sprint backlog items that have been chosen for that sprint. If it is impossible to complete all of the sprint backlog items, the time for the sprint is not extended. Rather, the unfinished items are returned to the product backlog and queued for a future sprint.
3. *Sprint reviewing* The work done during the sprint is reviewed by the team and (possibly) external stakeholders. The team reflects on what went well and what went wrong during the sprint, with a view to improving the work process.

Figure 2.7 shows the cycle of these activities and a more detailed breakdown of sprint execution. The sprint backlog is created during the planning process and drives the development activities when the sprint is executed.

Figure 2.7 Sprint activities

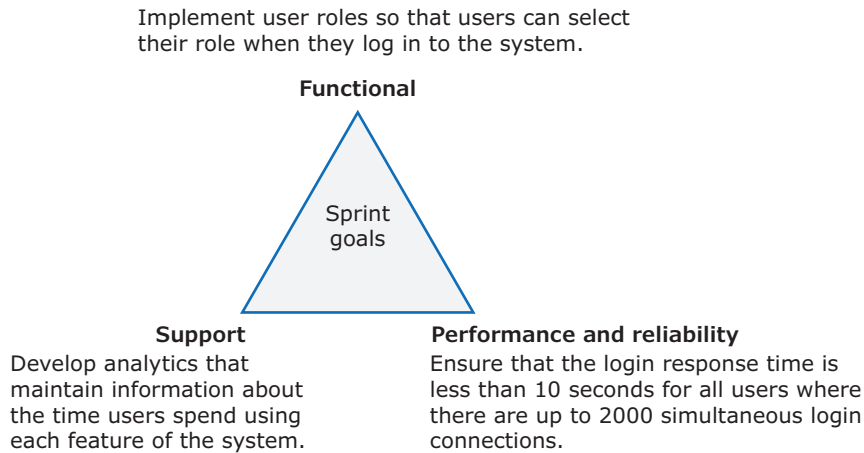
Each sprint should start with a planning meeting at which team members collectively decide on the PBIs to be implemented during the sprint. The inputs to this activity are the product backlog items that are ready for implementation and information from the Product Owner about which of these PBIs has the highest priority.

When planning a sprint, the team do three things:

- agree on a sprint goal;
- decide on the list of items from the product backlog that should be implemented;
- create a sprint backlog, a more detailed version of the product backlog that records the work to be done during the sprint.

The sprint goal is a succinct statement of what the team plans to achieve during a sprint. It could be the implementation of a product feature, the development of some essential product infrastructure, or the improvement of some product attribute, such as its performance. It should be possible to decide objectively whether or not the goal has been achieved by the end of the sprint. Figure 2.8 shows the three types of sprint goals and gives an example of each type.

Functional sprint goals relate to the implementation of system features for end-users. Performance and reliability goals relate to improvements in the performance, efficiency, reliability, and security of the system. Support goals cover ancillary activities such as developing infrastructure software or designing the system architecture.

Figure 2.8 Sprint goals

You should always consider the highest-priority items on the product backlog when deciding on a sprint goal. The team chooses these items for implementation at the same time as the sprint goal is being set. It makes sense to choose a coherent set of high-priority items that are consistent with the sprint goal. Sometimes items of lower priority in the product backlog are chosen because they are closely related to other items that are part of the overall sprint goal.

As a general rule, the sprint goal should not be changed during the sprint. Sometimes, however, the sprint goal has to be changed if unexpected problems are discovered or if the team finds a way to implement a feature more quickly than originally estimated. In these cases, the scope of the goal may be reduced or extended.

Once a sprint goal has been established, the team should discuss and decide on a sprint plan. As I explained, PBIs should have an associated effort estimate, which is a critical input to the sprint planning process. It's important that a team does not try to implement too many items during the sprint. Overcommitment may make it impossible to achieve the sprint goal.

The velocity of the team is another important input to the sprint planning process. The velocity reflects how much work the team can normally cover in a sprint. You may estimate story points as I have explained, where the team's velocity is the number of story points it can normally implement in a two-week or four-week sprint. This approach obviously makes sense for a team that has a stable velocity.

A team's velocity might be unstable, however, which means that the number of PBIs completed varies from one sprint to another. Velocity may

Table 2.8 Scrum

A scrum is a short, daily meeting that is usually held at the beginning of the day. During a scrum, all team members share information, describe their progress since the previous day's scrum, and present problems that have arisen and plans for the coming day. This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Scrum meetings should be short and focused. To dissuade team members from getting involved in long discussions, scrums are sometimes organized as “stand-up” meetings where there are no chairs in the meeting room.

During a scrum, the sprint backlog is reviewed. Completed items are removed from it. New items may be added to the backlog as new information emerges. The team then decides who should work on sprint backlog items that day.

be unstable if the team membership changes, if easier items are assigned a higher priority than items that are harder to implement, or if a team includes inexperienced members who improve as the project progresses. If a team's velocity is unstable or unknown, then you have to take a more intuitive approach to choosing the number of PBIs to be implemented during a sprint.

The sprint backlog is a list of work items to be completed during the sprint. Sometimes, a PBI can be transferred directly to the sprint backlog. However, the team normally breaks down each PBI into smaller tasks that are added to the sprint backlog. All team members then discuss how these tasks will be allocated. Each task should have a relatively short duration—one or two days at most—so that the team can assess its progress during the daily sprint meeting. The sprint backlog should be much shorter than the product backlog, so it can be maintained on a shared whiteboard. The whole team can see what items are to be implemented and what items have been completed.

The focus of a sprint is the development of product features or infrastructure and the team works to create the planned software increment. To facilitate cooperation, team members coordinate their work every day in a short meeting called a scrum (Table 2.8). The Scrum method is named after these meetings, which are an essential part of the method. They are a way for teams to communicate—nothing like scrums in the game of rugby.

The Scrum method does not include specific technical development practices; the team may use any agile practices they think are appropriate. Some teams like pair programming; others prefer that members work individually. However, I recommend that two practices always be used in code development sprints:

Table 2.9 Code completeness checklist

State	Description
Reviewed	The code has been reviewed by another team member who has checked that it meets agreed coding standards, is understandable, includes appropriate comments, and has been refactored if necessary.
Unit tested	All unit tests have been run automatically and all tests have executed successfully.
Integrated	The code has been integrated with the project codebase and no integration errors have been reported.
Integration tested	All integration tests have been run automatically and all tests have been executed successfully.
Accepted	Acceptance tests have been run if appropriate and the Product Owner or the development team has confirmed that the product backlog item has been completed.

1. *Test automation* As far as possible, product testing should be automated. You should develop a suite of executable tests that can be run at any time. I explain how to do this in Chapter 9.
2. *Continuous integration* Whenever anyone makes changes to the software components they are developing, these components should be immediately integrated with other components to create a system. This system should then be tested to check for unanticipated component interaction problems. I explain continuous integration in Chapter 10.

The aim of a sprint is to develop a “potentially shippable product increment.” Of course, the software will not necessarily be released to customers, but it should not require further work before it can be released. This means different things for different types of software, so it is important that a team establish a “definition of done,” which specifies what has to be completed for code that is developed during a sprint.

For example, for a software product that is being developed for external customers, the team may create a checklist that applies to all the software that is being developed. Table 2.9 is an example of a checklist that can be used to judge the completeness of an implemented feature.

If it is not possible to complete all items on this checklist during a sprint, the unfinished items should be added to the product backlog for future implementation. The sprint should never be extended to complete unfinished items.

At the end of each sprint, there is a review meeting that involves the whole team. This meeting has three purposes. First, it reviews whether or not the sprint has met its goal. Second, it sets out any new problems and issues that have emerged during the sprint. Finally, it is a way for a team to reflect on how they can improve the way they work. Members discuss what has gone well, what has gone badly, and what improvements could be made.

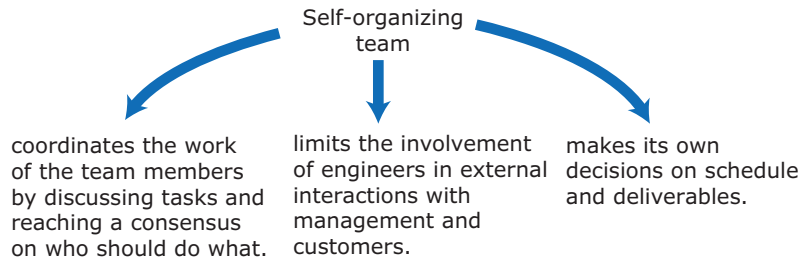
The review may involve external stakeholders as well as the development team. The team should be honest about what has and hasn't been achieved during the sprint so that the output of the review is a definitive assessment of the state of the product being developed. If items are unfinished or if new items have been identified, these should be added to the product backlog. The Product Owner has the ultimate authority to decide whether or not the goal of the sprint has been achieved. They should confirm that the implementation of the selected product backlog items is complete.

An important part of a sprint review is a process review, in which the team reflects on its own way of working and how Scrum has been used. The aim of a process review is to identify ways to improve and to discuss how to use Scrum more productively. Over the course of a development process, a Scrum team should try to continually improve its effectiveness.

During the review, the team may discuss communication breakdowns, good and bad experiences with tools and the development environment, technical practices that have been adopted, reusable software and libraries that have been discovered, and other issues. If problems have been identified, the team should discuss how they should be addressed in future sprints. For example, a decision may be made to investigate alternative tools to those being used by the team. If aspects of the work have been successful, the team may explicitly schedule time so that experience can be shared and good practice adopted across the team.

2.3.3 Self-organizing teams

A fundamental principle of all agile development methods is that the software development team should be self-organizing. Self-organizing teams don't have a project manager who assigns tasks and makes decisions for the team. Rather, as shown in Figure 2.9, they make their own decisions. Self-organizing teams work by discussing issues and making decisions by consensus.

Figure 2.9 Self-organizing teams

The ideal Scrum team size is between five and eight people—large enough to be diverse yet small enough to communicate informally and effectively and to agree on the priorities of the team. Because teams have to tackle diverse tasks, it's important to have a range of expertise in a Scrum team such as networking, user experience, database design and so on.

In reality, it may not be possible to form *ideal* teams. In a non-commercial setting such as a university, teams are smaller and made up of people who have largely the same skill set. There is a worldwide shortage of software engineers, so it is sometimes impossible to find people with the right mix of skills and experience. A team may change during a project as people leave and new members are hired. Some team members may work part-time or from home.

The advantage of an effective self-organizing team is that it can be cohesive and can adapt to change. Because the team rather than individuals takes responsibility for the work, the team can cope with people leaving and joining the group. Good team communication means that team members inevitably learn something about each other's areas. They can therefore compensate, to some extent, when people leave the team.

In a managed team, the project manager coordinates the work. Managers look at the work to be done and assign tasks to team members. Project managers have to arrange things so that work is not delayed because one team member is waiting for others to finish their work. They have to tell all team members about problems and other factors that may delay the work. Team members are not encouraged to take responsibility for coordination and communication.

In a self-organizing team, the team itself has to put in place ways to coordinate the work and communicate issues to all team members. The developers of Scrum assumed that team members are co-located. They work in the same

office and can communicate informally. If one team member needs to know something about what another has done, they simply talk to each other to find out. There is no need for people to document their work for others to read. Daily scrums mean that the team members know what's been done and what others are doing.

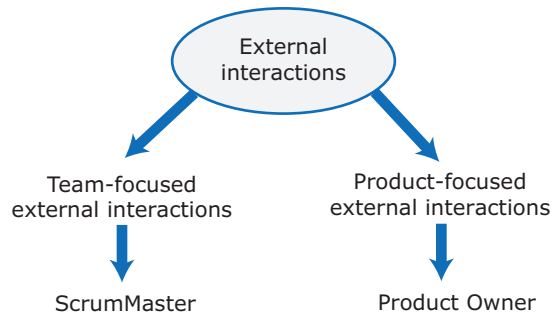
The Scrum approach embodies the essentials for coordination in a self-managed team—namely, good informal communication and regular meetings to ensure that everyone is up to speed. Team members explain their work and are aware of team progress and possible risks that may affect the team. However, there are practical reasons why informal verbal communication may not always work:

1. Scrum assumes that the team is made up of full-time workers who share a workspace. In reality, team members may be part-time and may work in different places. For a student project, team members may take different classes at different times, so it may be difficult to find a time slot where all team members can meet.
2. Scrum assumes that all team members can attend a morning meeting to coordinate the work for the day. This does not take into account that team members may work flexible hours (for example, because of child care responsibilities) or may work part-time on several projects. They are, therefore, not available every morning.

If co-located working with daily meetings is impractical, then the team must work out other ways to communicate. Messaging systems, such as Slack, can be effective for informal communications. The benefit of messaging is that all messages are recorded so that people can catch up on conversations that they missed. Messaging does not have the immediacy of face-to-face communication, but it is better than email or shared documents for coordination.

Talking to each other is the best way for team members to coordinate work and to communicate what has gone well and what problems have arisen. Daily meetings may be impossible, but agile teams really have to schedule progress meetings regularly even if all members can't attend or have to attend virtually using teleconferencing. Members who can't attend should submit a short summary of their own progress so that the team can assess how well the work is going.

All development teams, even those working in small startups or non-commercial developments, have some external interactions. Some interactions will help the team understand what customers require from the software

Figure 2.10 Managing external interactions

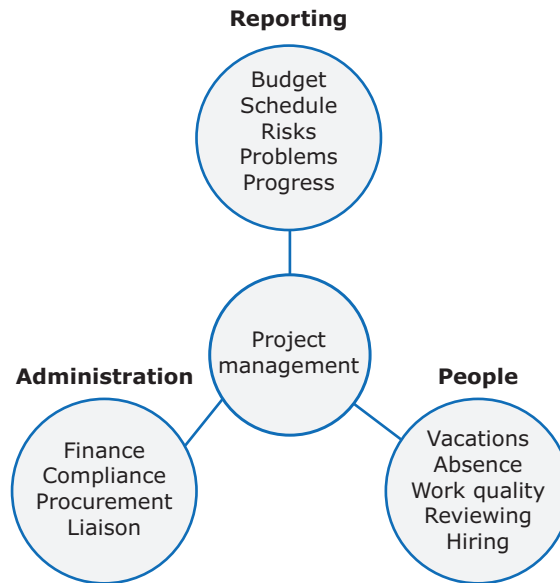
product being developed. Others will be with company management and other parts of the company, such as human resources and marketing.

In a Scrum project, the ScrumMaster and the Product Owner should be jointly responsible for managing interactions with people outside the team (Figure 2.10).

Product Owners are responsible for interactions with current and potential customers as well as the company's sales and marketing staff. Their job is to understand what customers are looking for in a software product and to identify possible barriers to the adoption and use of the product being developed. They should understand the innovative features of the product to establish how customers can benefit from them. Product Owners use this knowledge to help develop the product backlog and to prioritize backlog items for implementation.

The ScrumMaster role has a dual function. Part of the role is to work closely with the team, coaching them in the use of Scrum and working on the development of the product backlog. *The Scrum Guide* states that the ScrumMaster should also work with people outside of the team to “remove impediments”; that is, they should deal with external problems and queries and represent the team to the wider organization. The intention is for the team to be able to work on software development without external interference or distractions.

Whether or not a team is using Scrum or some other agile approach, you need to pay attention to these issues. In small teams, it may be impossible to have different people take care of interactions with customers and interactions with managers. The best approach may be for one person to take on both of these roles and to work part-time on software development. The key requirement for “external communicators” is good communication and people skills

Figure 2.11 Project management responsibilities

so that they can talk about the team's work in a way that people outside the team can understand and relate to.

The ScrumMaster is not a conventional project manager. The job is to help team members use the Scrum method effectively and to ensure that they are not distracted by external considerations. However, in all commercial projects, someone has to take on essential project management responsibilities (Figure 2.11).

The Scrum Guide and many Scrum books (although not Rubin's book that I've included in Recommended Reading) simply ignore these issues. But they are a reality of working in all but the smallest companies. A self-organizing team has to appoint a team member to take on management tasks. Because of the need to maintain continuity of communication with people outside of the group, sharing the management tasks among team members is not a viable approach.

In response to this issue, Rubin suggests that it may sometimes be appropriate for a project manager outside of the team to act for several Scrum teams. I think this idea is unworkable for three reasons:

1. Small companies may not have the resources to support dedicated project managers.

2. Many project management tasks require detailed knowledge of a team's work. If a project manager is working across several teams, it may be impossible to know the work of each team in detail.
3. Self-organizing teams are cohesive and tend to resent being told what to do by people outside of the team. Members are liable to obstruct, rather than support, an external project manager.

In my opinion it is unrealistic for the ScrumMaster role to exclude project management responsibilities. ScrumMasters know the work going on and are in by far the best position to provide accurate information and project plans and progress.

KEY POINTS

- The best way to develop software products is to use agile software engineering methods that are geared to rapid product development and delivery.
- Agile methods are based on iterative development and the minimization of overheads during the development process.
- Extreme Programming (XP) is an influential agile method that introduced agile development practices such as user stories, test-first development, and continuous integration. These are now mainstream software development activities.
- Scrum is an agile method that focuses on agile planning and management. Unlike XP, it does not define the engineering practices to be used. The development team may use any technical practices they consider appropriate for the product being developed.
- In Scrum, work to be done is maintained in a product backlog, a list of work items to be completed. Each increment of the software implements some of the work items from the product backlog.
- Sprints are fixed-time activities (usually two to four weeks) in which a product increment is developed. Increments should be potentially shippable; that is, they should not need further work before they are delivered.
- A self-organizing team is a development team that organizes the work to be done by discussion and agreement among team members.
- Scrum practices, such as the product backlog, sprints, and self-organizing teams, can be used in any agile development process, even if other aspects of Scrum are not used.

RECOMMENDED READING

Extreme Programming Explained This was the first book on XP and I think it's still the best one. It explains the approach from the perspective of one of its inventors, and his enthusiasm comes through very clearly in the book. (K. Beck and C. Andres, Addison-Wesley, 2004)

Essential Scrum: A practical guide to the most popular agile process This is a comprehensive and readable description of the 2011 version of the Scrum method. The diagrams are sometimes overcomplicated and a bit difficult to understand, but it is the best book on Scrum I have seen. (K. S. Rubin, Addison-Wesley, 2012)

The Scrum Guide This definitive guide to the Scrum method defines all the Scrum roles and activities. (K. Schwaber and J. Sutherland, 2013)

<http://www.scrumguides.org/docs/scrumguide/v1/scrum-guide-us.pdf>

"The Agile Mindset" This blog post argues for a flexible, pick-and-mix approach to Scrum practices rather than the inflexible model proposed in *The Scrum Guide*. I generally agree with what the author is saying here. (D. Thomas, 2014)

<http://blog.scottlogic.com/2014/09/18/the-agile-mindset.html>

"The Advantages and Disadvantages of Agile Scrum Software Development" An article by a project management expert rather than a Scrum evangelist presents a balanced picture of the advantages and disadvantages of Scrum. (S. de Sousa, undated)

<http://www.my-project-management-expert.com/the-advantages-and-disadvantages-of-agile-scrum-software-development.html>

"A Criticism of Scrum" This tongue-in-cheek blog post sets out what the author really doesn't like about Scrum. I agree with some of his points; others I think are exaggerated. (A. Gray, 2015)

<https://www.aaron-gray.com/a-criticism-of-scrum/>

PRESENTATIONS, VIDEOS, AND LINKS

<https://iansommerville.com/engineering-software-products/agile-software-engineering>

EXERCISES

- 2.1. Explain why it is important that software products are developed and delivered quickly. Why is it sometimes sensible to deliver an unfinished product and then issue new versions of that product after delivery?
- 2.2. Explain why the fundamental objectives of agile software engineering are consistent with the accelerated development and delivery of software products.
- 2.3. Give three reasons why Extreme Programming, as envisaged by its developers, is not widely used.
- 2.4. You are developing a software product to help manage student admissions at a university. Your agile development team suggests that they create a number of small releases that potential customers can try and then provide feedback. Comment on this idea and suggest why it may not be acceptable to the system's users.
- 2.5. Explain why the Product Owner plays an essential role in a Scrum development team. How might a development team that is working in an environment where there are no external customers (e.g., a student project team) reproduce this Product Owner role?
- 2.6. Why is it important that each sprint normally produces a potentially shippable product increment? When might the team relax this rule and produce something that is not ready to ship?
- 2.7. Explain why estimating the effort required to complete a product backlog item using person-hours or person-days may lead to significant variations between the estimated effort and the actual effort.
- 2.8. Why are daily scrums likely to reduce the time that is normally required for new team members to become productive?
- 2.9. One problem with self-organizing teams is that more experienced team members tend to dominate discussions and therefore influence the team's way of working. Suggest ways to counteract this problem.
- 2.10. Scrum is designed for use by a team of five to eight people working together to develop a software product. What problems might arise if you try to use Scrum for student team projects in which members work together to develop a program? What parts of Scrum could be used in this situation?

3

Features, Scenarios, and Stories

Some software products are inspired. The developers of these products have a vision of the software that they want to create. They don't have a product manager, they don't do user surveys, they don't collect and document requirements or model how users will interact with the system. They simply get on with developing a prototype system. Some of the most successful software products, such as Facebook, started like this.

However, the vast majority of software products that are solely based on a developer's inspiration are commercial failures. These products either don't meet a real user need or don't fit with the ways in which users really work. Inspiration is important but most successful products are based on an understanding of business and user problems and user interaction. Even when inspiration leads to many users adopting a product, continuing use depends on its developers understanding how the software is used and new features that users may want.

Apart from inspiration, there are three factors that drive the design of software products:

1. *Business and consumer needs that are not met by current products* For example, book and magazine publishers are moving to both online and paper publication, yet few software products allow seamless conversion from one medium to another.
2. *Dissatisfaction with existing business or consumer software products* For example, many current software products are bloated with features that are rarely used. New companies may decide to produce simpler products in the same area that meet the needs of most users.

3. *Changes in technology that make completely new types of products possible* For example, as virtual reality (VR) technology matures and the hardware gets cheaper, new products may exploit this opportunity.

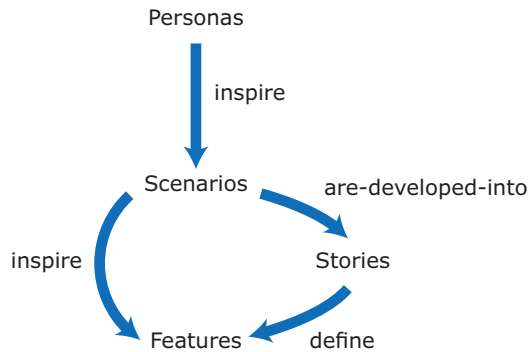
As I explained in Chapter 1, software products are not developed to meet the requirements of a specific client. Consequently, techniques that have been developed for eliciting, documenting, and managing software requirements aren't used for product engineering. You don't need to have a complete and detailed requirements document as part of a software development contract. There is no need for prolonged consultations when requirements change. Product development is incremental and agile, so you can use less formal ways of defining your product.

In the early stage of product development, rather than understanding the requirements of a specific client, you are trying to understand what product features will be useful to users and what they like and dislike about the products that they use. Briefly, a feature is a fragment of functionality, such as a Print feature, a Change Background feature, a New Document feature, and so on. Before you start programming a product, you should create a list of features to be included in your product. This is your starting point for product design and development.

It makes sense in any product development to spend time trying to understand the potential users and customers of your product. A range of techniques have been developed for understanding the ways that people work and use software. These include user interviews, surveys, ethnography, and task analysis.¹ Some of these techniques are expensive and unrealistic for small companies. However, informal user analysis and discussions, which simply involve asking users about their work, the software that they use, and its strengths and weaknesses, are inexpensive and very valuable.

One problem with informal user studies for business products is that the users simply may not want new software. For business products, the business buys the product, but its employees are the users. These users may be hostile to new products because they have to change their familiar way of working or perhaps because increased automation may reduce the number of jobs available. Business managers may suggest what they want from a new software product, but this does not always reflect the needs or wishes of the product's users.

¹I discuss techniques of user analysis to discover software requirements in my general software engineering textbook, *Software Engineering*, 10th edition (Pearson Education, 2015).

Figure 3.1 From personas to features

In this chapter, I assume that informal user consultations are possible. I explain ways of representing users (personas) and communicating with them and other product stakeholders. I focus on how short, natural language descriptions (scenarios and stories) can be used to visualize and document how users might interact with a software product.

Figure 3.1 shows that personas, scenarios, and user stories lead to features that might be implemented in a software product.

If you look on the web, you can find a range of definitions of a “product feature,” but I think of a feature as a fragment of functionality that implements some user or system need. You access features through the user interface of a product. For example, the editor that I used to write this book includes a feature to create a “New Group,” in which a group is a set of documents that is accessed as a pull-down menu.

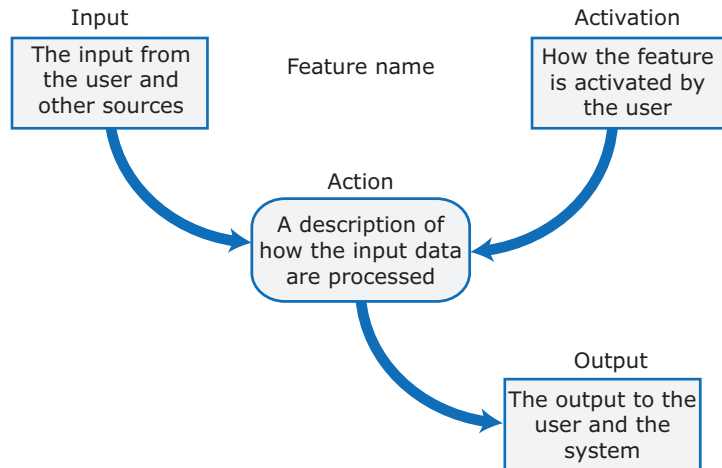
A feature is something that the user needs or wants. You can write a user story to make this explicit:

As an author I need a way to organize the text of the book that I’m writing into chapters and sections.

Using the New Group feature, you create a group for each chapter, and the documents within that group are the sections of that chapter. This feature can be described using a short, narrative, feature description:

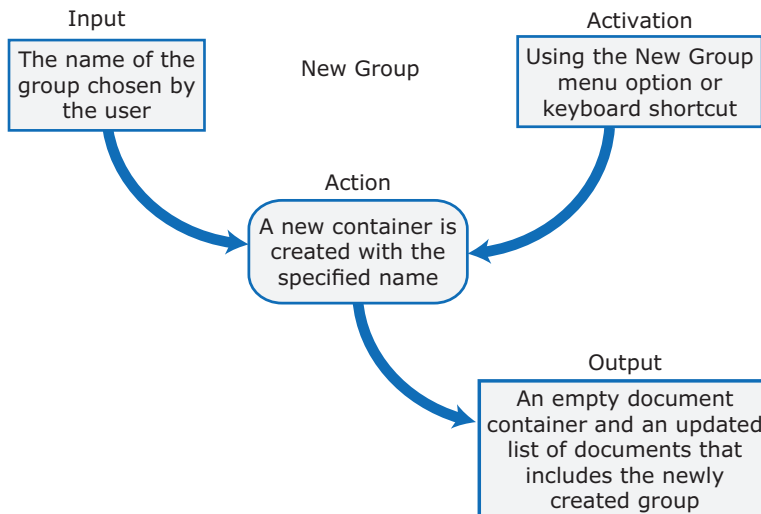
The “New Group” command, activated by a menu choice or keyboard shortcut, creates a named container for a set of documents and groups.

Alternatively, you can use a standard template where you define the feature by its input, its functionality, its output, and how it is activated. Figure 3.2

Figure 3.2 Feature description

shows the elements of the standard template, and Figure 3.3 shows how the New Group feature can be defined using this template.

Features can be defined using the input/action/output model that I have shown. However, system developers often need only a short feature description; then they fill in the feature details. This is especially true when the

Figure 3.3 The New Group feature description

feature is a “utility” feature that is commonly available in other products. For example, the Cut and Paste feature is well known, so might be defined as:

Cut and Paste – any selected object can be cut or copied, then inserted elsewhere into the document.

Sometimes all you need to say is that a Cut and Paste feature should be included and then rely on the developer’s general understanding to implement this.

Features are the fundamental elements of an agile method called Feature-Driven Development (FDD). I have no experience with this method and have not met anyone who uses it for product development, so I can’t comment on it directly. One aspect of this approach that I have used, however, is its template for feature description:

*<action> **the** <result> <by/for/lofto> <object>*

So, the New Group feature above could be described as:

*Create **the** container **for** documents or groups*

I show another example of this simple approach to feature description in Section 3.4.2.

I return to the topic of features in Section 3.4 after I have described scenarios and user stories. You can use both of these narrative techniques for deriving the list of features to be included in a system.

3.1 Personas

One of the first questions you should ask when developing a software product is “Who are the target users for my product?” You need to have some understanding of your potential users in order to design features that they are likely to find useful and to design a user interface that is suited to them.

Sometimes you know the users. If you are a software engineer developing software tools for other engineers, then you understand, to some extent at least, what they are likely to want. If you are designing a phone or tablet app for general use, you may talk to friends and family to understand what potential users may like or dislike. In those circumstances, you may be able to design using an

Table 3.1 A persona for a primary school teacher

Jack, a primary school teacher
Jack, age 32, is a primary school (elementary school) teacher in Ullapool, a large coastal village in the Scottish Highlands. He teaches children from ages 9 to 12. He was born in a fishing community north of Ullapool, where his father runs a marine fuels supply business and his mother is a community nurse. He has a degree in English from Glasgow University and retrained as a teacher after several years working as a web content author for a large leisure group. Jack's experience as a web developer means that he is confident in all aspects of digital technology. He passionately believes that the effective use of digital technologies, blended with face-to-face teaching, can enhance the learning experience for children. He is particularly interested in using the iLearn system for project-based teaching, where students work together across subject areas on a challenging topic.

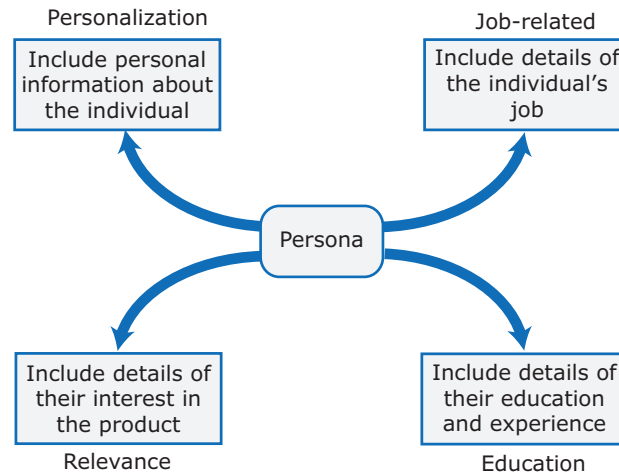
intuitive idea of who the users are and what they can do. However, you should be aware that your users are diverse and your own experience may not provide a complete picture of what they might want and how they might work.

For some types of software products, you may not know much about the background, skills, and experience of potential users. Individuals on the development team may have their own ideas about the product users and their capabilities. This can lead to product inconsistencies as these different views are reflected in the software implementation. Ideally, the team should have a shared vision of users, their skills, and their motivations for using the software. Personas are one way of representing this shared vision.

Personas are about “imagined users,” character portraits of types of user that you think might adopt your product. For example, if your product is aimed at managing appointments for dentists, you might create a dentist persona, a receptionist persona, and a patient persona. Personas of different types of users help you imagine what these users may want to do with your software and how they might use it. They also help you envisage difficulties that users might have in understanding and using product features.

A persona should paint a picture of a type of product user. You should describe the users' backgrounds and why they might want to use your product. You should also say something about their education and technical skills. This helps you assess whether or not a software feature is likely to be useful and understandable by typical product users.

An example of a persona that I wrote when designing the iLearn system, described in Chapter 1, is shown in Table 3.1. This is the persona of a teacher who is committed to digital learning and who believes that using digital devices can enhance the overall learning process.

Figure 3.4 Persona descriptions

There is no standard way of representing a persona; if you search the web, you will find a number of different recommendations. Common features of these suggestions are shown in Figure 3.4. The recommended aspects of a persona description—namely, personalization, relevance, education, and job-related—are explained in Table 3.2.

Many recommendations about what should be included in a persona suggest describing the individual's goals. I disagree because it's impossible to pin down what is meant by "goals." Goals are a broad concept. Some people have goals of self-improvement and learning; others have work-oriented goals of career progression and promotion. For some people, goals are not related to work. Their goals are simply to get through the day and earn enough so that they can do the things outside of work that give them pleasure and satisfaction.

My experience is that trying to define "user goals" is not helpful. I have found that most people don't have clearly defined goals when they use software. They may have been told to use the software as part of their job, they may see the software as a way to do their work more effectively, or they may find the software useful in organizing their life. Rather than try to set out goals, I think it's better to try to explain why the software might be useful and to give examples of what potential users may want to do with it.

If your product is targeted at a specific group of users, you may need only one or two personas to represent potential system users. For some products, however, the user group may be very broad and you may think that a large number of personas are needed. In fact, using many personas can make it more difficult to design a coherent system because they inevitably overlap.

Table 3.2 Aspects of persona descriptions

Aspect	Description
Personalization	You should give them a name and say something about their personal circumstances. It is sometimes helpful to use an appropriate stock photograph to represent the person in the persona. Some studies suggest that this helps project teams use personas more effectively.
Job-related	If your product is targeted at business, you should say something about their job and (if necessary) what that job involves. For some jobs, such as a teacher where readers are likely to be familiar with the job, this may not be necessary.
Education	You should describe their educational background and their level of technical skills and experience. This is important, especially for interface design.
Relevance	If you can, you should say why they might be interested in using the product and what they might want to do with it.

In general, you don't need more than five personas to help identify the key features of a system.

Personas should be relatively short and easy to read. For the iLearn system, we developed personas that we described in two or three paragraphs of text. We found that these had enough information to be useful. Two of the personas that we created are shown in Tables 3.3 and 3.4.

The persona in Table 3.3 represents users who do not have a technical background. They simply want a system to provide support for administration.

Table 3.3 A persona for a history teacher

Emma, a history teacher
Emma, age 41, is a history teacher in a secondary school (high school) in Edinburgh. She teaches students from ages 12 to 18. She was born in Cardiff in Wales, where both her father and her mother were teachers. After completing a degree in history from Newcastle University, she moved to Edinburgh to be with her partner and trained as a teacher. She has two children, aged 6 and 8, who both attend the local primary school. She likes to get home as early as she can to see her children, so often does lesson preparation, administration, and marking from home. Emma uses social media and the usual productivity applications to prepare her lessons, but is not particularly interested in digital technologies. She hates the virtual learning environment that is currently used in her school and avoids using it if she can. She believes that face-to-face teaching is most effective. She might use the iLearn system for administration and access to historical films and documents. However, she is not interested in a blended digital/face-to-face approach to teaching.

Table 3.4 A persona for an IT technician

Elena, a school IT technician
Elena, age 28, is a senior IT technician in a large secondary school (high school) in Glasgow with over 2000 students. Originally from Poland, she has a diploma in electronics from Potsdam University. She moved to Scotland in 2011 after being unemployed for a year after graduation. She has a Scottish partner, no children, and hopes to develop her career in Scotland. She was originally appointed as a junior technician but was promoted, in 2014, to a senior post responsible for all the school computers. Although not involved directly in teaching, Elena is often called on to help in computer science classes. She is a competent Python programmer and is a “power user” of digital technologies. She has a long-term career goal of becoming a technical expert in digital learning technologies and being involved in their development. She wants to become an expert in the iLearn system and sees it as an experimental platform for supporting new uses for digital learning.

Elena’s persona in Table 3.4 represents technically skilled support staff who may be responsible for setting up and configuring the iLearn software.

I haven’t included personas for the students who were intended to be the ultimate end-users of the iLearn system. The reason is that we saw the iLearn system as a platform product that should be configured to suit the preferences and needs of individual schools and teachers. Students would use iLearn to access tools. However, they would not use the configuration features that make iLearn a unique system. Although we planned to include a standard set of applications with the system, we were driven by the belief that the best people to create learning systems were teachers, supported by local technical staff.

Ideally, software development teams should be diverse, with people of different ages and genders. However, the reality is that software product developers are still, overwhelmingly, young men with a high level of technical skill. Software users are more diverse, with varying levels of technical skill. Some developers find it hard to appreciate the problems that users may have with the software. An important benefit of personas is that they help the development team members empathize with potential users of the software. Personas are a tool that allows team members to “step into the users’ shoes.” Instead of thinking about what they would do in a particular situation, they can imagine how a persona would behave and react.

So, when you have an idea for a feature, you can ask “Would this persona be interested in this feature?” and “How would that persona access and use the feature?”. Personas can help you check your ideas to make sure that you are not including product features that aren’t really needed. They help you to avoid making unwarranted assumptions, based on your own knowledge, and designing an overcomplicated or irrelevant product.

Personas should be based on an understanding of the potential product users: their jobs, their backgrounds, and their aspirations. You should study and survey potential users to understand what they want and how they might use the product. From these data, you can abstract the essential information about the different types of product users and then use this as a basis for creating personas. These personas should then be cross-checked against the user data to make sure that they reflect typical product users.

It may be possible to study the users when your product is a development of an existing product. For new products, however, you may find it impractical to carry out detailed user surveys. You may not have easy access to potential users. You may not have the resources to carry out user surveys, and you may want to keep your product confidential until it is ready to launch.

If you know nothing about an area, you can't develop reliable personas, so you need to do some user research before you start. This does not have to be a formal or prolonged process. You may know people working in an area and they might be able to help you meet their colleagues to discuss your ideas. For example, my daughter is a teacher and she helped arrange lunch with a group of her colleagues to discuss how they use digital technologies. This helped with both persona and scenario development.

Personas that are developed on the basis of limited user information are called proto-personas. Proto-personas may be created as a collective team exercise using whatever information is available about potential product users. They can never be as accurate as personas developed from detailed user studies, but they are better than nothing. They represent the product users as seen by the development team, and they allow the developers to build a common understanding of the potential product users.

3.2 Scenarios

As a product developer, your aim should be to discover the product features that will tempt users to adopt your product rather than competing software. There is no easy way to define the “best” set of product features. You have to use your own judgement about what to include in your product. To help select and design features, I recommend that you invent scenarios to imagine how users could interact with the product that you are designing.

A scenario is a narrative that describes a situation in which a user is using your product's features to do something that they want to do. The scenario should briefly explain the user's problem and present an imagined way that

Table 3.5 Jack’s scenario: Using the iLearn system for class projects

Fishing in Ullapool
Jack is a primary school teacher in Ullapool, teaching P6 pupils. He has decided that a class project should be focused around the fishing industry in the area, looking at the history, development, and economic impact of fishing. As part of this, students are asked to gather and share reminiscences from relatives, use newspaper archives, and collect old photographs related to fishing and fishing communities in the area. Pupils use an iLearn wiki to gather together fishing stories and SCRAN (a history archive site) to access newspaper archives and photographs. However, Jack also needs a photo-sharing site as he wants students to take and comment on each others’ photos and to upload scans of old photographs that they may have in their families. He needs to be able to moderate posts with photos before they are shared, because pre-teen children can’t understand copyright and privacy issues. Jack sends an email to a primary school teachers’ group to see if anyone can recommend an appropriate system. Two teachers reply and both suggest that he use KidsTakePics, a photo-sharing site that allows teachers to check and moderate content. As KidsTakePics is not integrated with the iLearn authentication service, he sets up a teacher and a class account with KidsTakePics. He uses the the iLearn setup service to add KidsTakePics to the services seen by the students in his class so that, when they log in, they can immediately use the system to upload photos from their phones and class computers.

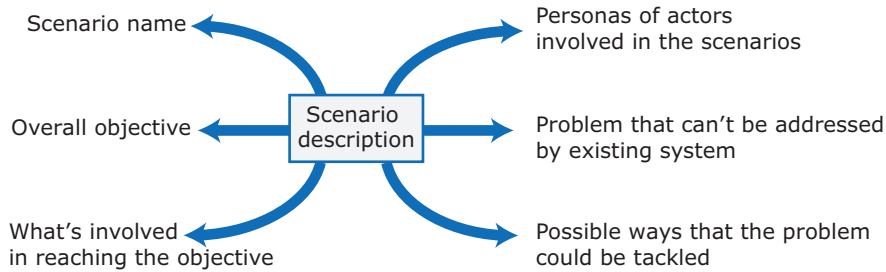
the problem might be solved. There is no need to include everything in the scenario; it isn’t a detailed system specification.

Table 3.5 is an example scenario that shows how Jack, whose persona I described in Section 3.2, might use the iLearn system.

From this description of how iLearn might be used for class projects, you can see some of the key elements of a scenario (Figure 3.5) that may be included to help you think about the product features that you need.

The most important elements of a scenario are:

1. A brief statement of the overall objective. In Jack’s scenario, shown in Table 3.5, this is to support a class project on the fishing industry.
2. References to the persona involved (Jack) so that you can get information about the capabilities and motivation of that user.
3. Information about what is involved in doing the activity. For example, in Jack’s scenario, this involves gathering reminiscences from relatives, accessing newspaper archives, and so on.
4. If appropriate, an explanation of problems that can’t be readily addressed using the existing system. Young children don’t understand

Figure 3.5 Elements of a scenario description

issues such as copyright and privacy, so photo sharing requires a site that a teacher can moderate to make sure that published images are legal and acceptable.

5. A description of one way that the identified problem might be addressed. This may not always be included especially if technical knowledge is needed to solve the problem. In Jack's scenario, the preferred approach is to use an external tool designed for school students.

The idea of using scenarios to support software engineering has been around since the 1980s. Different types of scenarios have been proposed, ranging from high-level scenarios like this one about Jack to more detailed and specific scenarios that set out the steps involved in a user's interaction with a system. They are the basis for both use cases, which are extensively used in object-oriented methods, and user stories, which are used in agile methods. Scenarios are used in the design of requirements and system features, in system testing, and in user interface design.

Narrative, high-level scenarios, like Jack's scenario, are primarily a means of facilitating communication and stimulating design creativity. They are effective in communication because they are understandable and accessible to users and to people responsible for funding and buying the system.² Like personas, they help developers to gain a shared understanding of the system that they are creating. You should always be aware, however, that scenarios are not specifications. They lack detail, they may be incomplete, and they may not represent all types of user interactions.

²I presented some of the scenarios for the iLearn system to a government minister for education. He commented that this was the first time he had ever attended a meeting on an IT system where he actually understood what the system was supposed to do.

Some people recommend that scenarios should be structured with different fields such as what the user sees at the beginning of a scenario, a description of the normal flow of events, a description of what might go wrong, and so on. If you are using scenarios to elicit detailed requirements, the benefit of structure is that you have a consistent presentation, which means you are less likely to forget to include elements that relate to critical system requirements. Engineers usually like this structured approach. However, my experience is that system users, who read and check the scenarios, find structured scenarios to be intimidating and hard to understand.

Consequently, when you are using scenarios at the early stages of product design, I recommend narrative rather than structured scenarios. These scenarios may range from two or three paragraphs of text, like Jack's scenario in Table 3.5, to longer descriptions. You may need to write longer descriptions when your software will be used in existing processes and must interoperate with other software. Your scenarios may then have to include descriptions of interactions with other processes and software systems.

Emma's scenario (shown in Table 3.6) is an example of a longer scenario. In it, she uses the iLearn system to help her with the administration involved in setting up a class trip.

Emma's scenario is different from Jack's scenario because it describes a common and well-understood process rather than something new. The scenario discusses how parts of the process (setting up an email group and web page) are automated by the iLearn system. Remember that Emma is an e-learning skeptic; she is not interested in innovative applications. She wants a system that will make her life easier and reduce the amount of routine administration that she has to do.

In this type of scenario, you are much more likely to include specific details of what might be in the system. For example, it explains that Emma logs in to the system with her Google credentials. This means she doesn't have to remember a separate login name and password. I discuss this approach to authentication in Chapter 7.

When you see this kind of information in a scenario, you need to check whether this is what the user really needs or whether it represents a more general requirement. The statement that the software has to support login using Google credentials might actually reflect a more general need—to provide a login mechanism in which users don't have to remember yet another set of credentials. You may decide on an alternative approach to authentication, such as fingerprint or face recognition on a mobile phone, that avoids the need for system-specific login credentials.

Table 3.6 Using the iLearn system for administration

Emma is teaching the history of World War I to a class of 14-year-olds (S3). A group of S3 students are visiting the historic World War I battlefields in northern France. She wants to set up a “battlefields group” where the students who are attending the trip can share their research about the places they are visiting as well as their pictures and thoughts about the visit.

From home, she logs onto the iLearn system using her Google account credentials. Emma has two iLearn accounts—her teacher account and a parent account associated with the local primary school. The system recognizes that she is a multiple account owner and asks her to select the account to be used. She chooses the teacher account and the system generates her personal welcome screen. As well as her selected applications, this also shows management apps that help teachers create and manage student groups.

Emma selects the “group management” app, which recognizes her role and school from her identity information and creates a new group. The system prompts for the class year (S3) and subject (history) and automatically populates the new group with all S3 students who are studying history. She selects those students going on the trip and adds her teacher colleagues, Jamie and Claire, to the group.

She names the group and confirms that it should be created. The app sets up an icon on her iLearn screen to represent the group, creates an email alias for the group, and asks Emma if she wishes to share the group. She shares access with everyone in the group, which means that they also see the icon on their screen. To avoid getting too many emails from students, she restricts sharing of the email alias to Jamie and Claire.

The group management app then asks Emma if she wishes to set up a group web page, wiki, and blog. Emma confirms that a web page should be created and she types some text to be included on that page.

She then accesses Flickr using the icon on her screen, logs in, and creates a private group to share trip photos that students and teachers have taken. She uploads some of her own photos from previous trips and emails an invitation to join the photo-sharing group to the battlefields email list. Emma uploads material from her own laptop that she has written about the trip to iLearn and shares this with the battlefields group. This action adds her documents to the web page and generates an alert to group members that new material is available.

3.2.1 Writing scenarios

Your starting point for scenario writing should be the personas you have created. You should normally try to imagine several scenarios for each persona. Remember that these scenarios are intended to stimulate thinking rather than provide a complete description of the system. You don’t need to cover everything you think users might do with your product.

Scenarios should always be written from the user’s perspective and should be based on identified personas or real users. Some writers suggest that scenarios should focus on goals—what the user wants to do—rather than the mechanisms they use to do this. They argue that scenarios should not include

specific details of an interaction as these limit the freedom of feature designers. I disagree. As you can see from Emma's scenario, it sometimes makes sense to talk about mechanisms, such as login with Google, that both product users and developers understand.

Furthermore, writing scenarios in a general way that doesn't make assumptions about implementation can be potentially confusing for both users and developers. For example, I think that "X cuts paragraphs from the newspaper archive and pastes them into the project wiki" is easier to read and understand than "X uses an information transfer mechanism to move paragraphs from the newspaper archive to the project wiki."

Sometimes there may be a specific requirement to include a particular feature in the system because that feature is widely used. For example, Jack's scenario in Table 3.5 discusses the use of an iLearn wiki. Many teachers currently use wikis to support group writing and they specifically want to have wikis in the new system. Such user requirements might be even more specific. For example, when designing the iLearn system, we discovered that teachers wanted Wordpress blogs, not just a general blogging facility. So, you should include specific details in a scenario when they reflect reality.

Scenario writing is not a systematic process and different teams approach it in different ways. I recommend that each team member be given individual responsibility for creating a small number of scenarios and work individually to do this. Obviously, members may discuss the scenarios with users and other experts, but this is not essential. The team then discusses the proposed scenarios. Each scenario is refined based on that discussion.

Because it is easy for anyone to read and understand scenarios, it is possible to get users involved in their development. For the iLearn system, we found that the best approach was to develop an imaginary scenario based on our understanding of how the system might be used and then ask users to tell us what we got wrong. Users could ask about things they did not understand, such as "Why can't a photo-sharing site like Flickr be used in Jack's scenario?" They could suggest how the scenario could be extended and made more realistic.

We tried an experiment in which we asked a group of users to write their own scenarios about how they might use the system. This was not a success. The scenarios they created were simply based on how they worked at the moment. They were far too detailed and the writers couldn't easily generalize their experience. Their scenarios were not useful because we wanted something to help us generate ideas rather than replicate the systems that they already used.

Scenarios are not software specifications but are ways of helping people think about what a software system should do. There is no simple answer

Table 3.7 Elena’s scenario: Configuring the iLearn system

Elena has been asked by David, the head of the art department in her school, to help set up an iLearn environment for his department. David wants an environment that includes tools for making and sharing art, access to external websites to study artworks, and “exhibition” facilities so that the students’ work can be displayed.

Elena starts by talking to art teachers to discover the tools that they recommend and the art sites that they use for studies. She also discovers that the tools they use and the sites they access vary according to the age of their students. Consequently, different student groups should be presented with a toolset that is appropriate for their age and experience.

Once she has established what is required, Elena logs into the iLearn system as an administrator and starts configuring the art environment using the iLearn setup service. She creates sub-environments for three age groups plus a shared environment that includes tools and sites that may be used by all students.

She drags and drops tools that are available locally and the URLs of external websites into each of these environments. For each of the sub-environments, she assigns an art teacher as its administrator so that they can’t refine the tool and website selection that has been set up. She publishes the environments in “review mode” and makes them available to the teachers in the art department.

After discussing the environments with the teachers, Elena shows them how to refine and extend the environments. Once they have agreed that the art environment is useful, it is released to all students in the school.

to the question “How many scenarios do I need?” In the iLearn system, 22 scenarios were developed to cover different aspects of system use. There was quite a lot of overlap among these scenarios, so it was probably more than we really needed. Generally, I recommend developing three or four scenarios per persona to get a useful set of information.

Although scenarios certainly don’t have to describe every possible use of a system, it is important that you look at the roles of each of the personas that you have developed and write scenarios that cover the main responsibilities for that role. Jack’s scenario and Emma’s scenario are based on using the iLearn system to support teaching.

Like other system products designed for use in an organization, however, iLearn needs to be configured for use. While some of this configuration can be done by tech-savvy teachers, in many schools it is technical support staff who have this responsibility. Elena’s scenario, shown in Table 3.7, describes how she might configure the iLearn software.

Writing scenarios always gives you ideas for the features that you can include in the system. You may then develop these ideas in more detail by analyzing the text of the scenario, as I explain in the next section.

3.3 User stories

I explained in Section 3.2 that scenarios are descriptions of situations in which a user is trying to do something with a software system. Scenarios are high-level stories of system use. They should describe a sequence of interactions with the system but should not include details of these interactions.

User stories are finer-grain narratives that set out in a more detailed and structured way a single thing that a user wants from a software system. I presented a user story at the beginning of the chapter:

As an author I need a way to organize the book that I'm writing into chapters and sections.

This story reflects what has become the standard format of a user story:

As a <role>, I <want / need> to <do something>

Another example of a user story taken from Emma's scenario might be:

As a teacher, I want to tell all members of my group when new information is available.

A variant of this standard format adds a justification for the action:

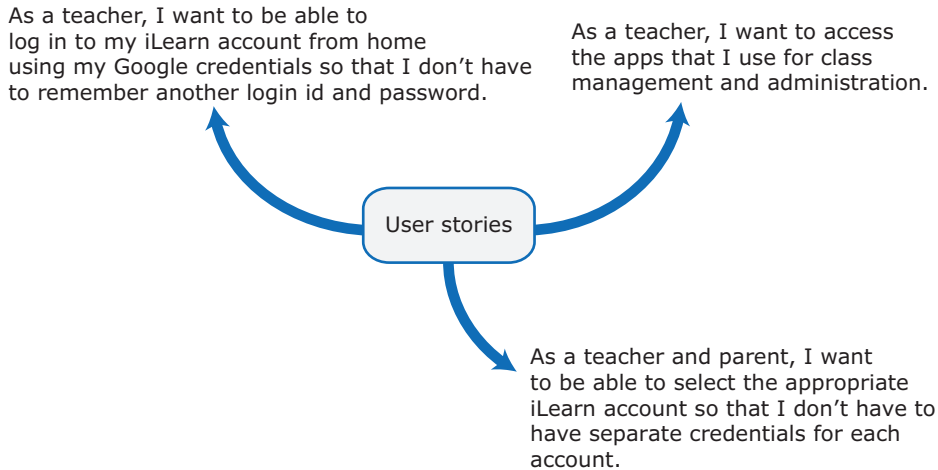
As a <role> I <want / need> to <do something> so that <reason>

For example:

As a teacher, I need to be able to report who is attending a class trip so that the school maintains the required health and safety records.

Some people argue that a rationale or justification should always be part of a user story. I think this is unnecessary if the story makes sense on its own. Knowing one reason why this might be useful doesn't help the product developers. However, in situations where some developers are unfamiliar with what users do, a rationale can help those developers understand why the story has been included. A rationale may also help trigger ideas about alternative ways of providing what the user wants.

An important use of user stories is in planning, and many users of the Scrum method represent the product backlog as a set of user stories. For this purpose, user stories should focus on a clearly defined system feature or

Figure 3.6 User stories from Emma’s scenario

aspect of a feature that can be implemented within a single sprint. If the story is about a more complex feature that might take several sprints to implement, then it is called an “epic.” An example of an epic might be:

As a system manager, I need a way to back up the system and restore individual applications, files, directories, or the whole system.

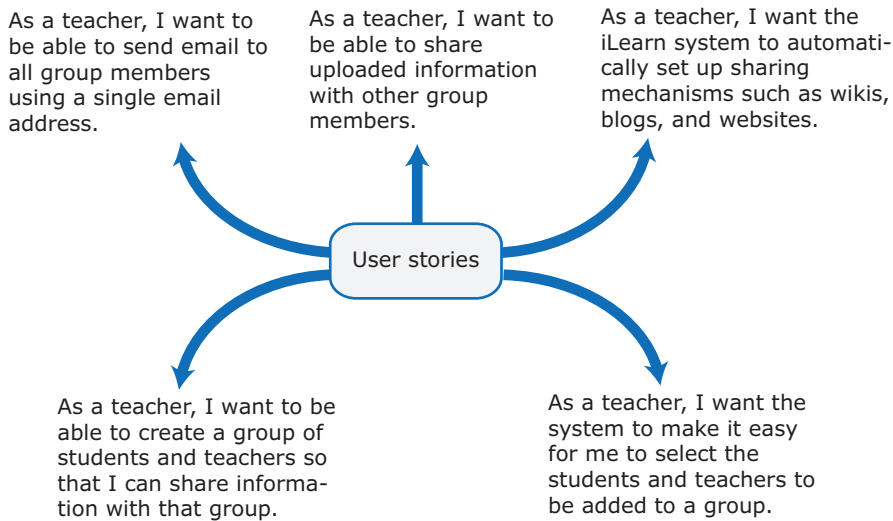
A lot of functionality is associated with this user story. For implementation, it should be broken down into simpler stories, with each story focusing on a single aspect of the backup system.

When you are thinking about product features, user stories are not intended for planning but for helping with feature identification. Therefore, you don’t need to be overly concerned about whether your stories are simple stories or epics. You should aim to develop stories that are helpful in one of two ways:

- as a way of extending and adding detail to a scenario;
- as part of the description of the system feature that you have identified.

As an example of scenario refinement, the initial actions in Emma’s scenario shown in Figure 3.6 can be represented by three user stories. Recall that the scenario says:

From home, she logs onto the iLearn system using her Google account credentials. Emma has two iLearn accounts—her teacher account and

Figure 3.7 User stories describing the Groups feature

a parent account associated with the local primary school. The system recognizes that she is a multiple account owner and asks her to select the account to be used. She chooses the teacher account and the system generates her personal welcome screen. As well as her selected applications, this also shows management apps that help teachers create and manage student groups.

You can create user stories from this account, as I show in Figure 3.6.

You can see from the stories in Figure 3.6 that I have included a rationale in the story that explains why Emma wants to work in the way specified in the scenario. It is best not to include rationale in a scenario, as it tends to disrupt the flow of the description and make it more difficult to read and understand.

When you define user stories from a scenario, you provide more information to developers to help them design the product's features. We can see an example of this in the stories shown in Figure 3.6. Emma wants an easy way to authenticate herself to the system, either as a teacher or as a parent. She doesn't want to have to remember more login credentials, and she doesn't want to have two accounts with different credentials.

As an example of how stories can be used to describe the features of a system, Emma's scenario discusses how a group can be created and explains system actions that take place on group creation. The user stories shown in Figure 3.7 can be derived from the scenario to describe the Groups feature in the iLearn system.

The set of stories shown in Figure 3.7 is not a complete description of the Groups feature. No stories are concerned with deleting or changing a group, restricting access, and other tasks. You start by deriving stories from a scenario, but you then have to think about what other stories might be needed for a complete description of a feature's functionality.

A question that is sometimes asked about user stories is whether you should write “negative stories” that describe what a user doesn't want. For example, you might write this negative story:

As a user, I don't want the system to log and transmit my information to any external servers.

If you are writing stories to be part of a product backlog, you should avoid negative stories. It is impossible to write system tests that demonstrate a negative. In the early stages of product design, however, it may be helpful to write negative stories if they define an absolute constraint on the system. Alternatively, you can sometimes reframe negative stories in a positive way. For example, instead of the above story, you could write:

As a user, I want to be able to control the information that is logged and transmitted by the system to external servers so that I can ensure that my personal information is not shared.

Some of the user stories that you develop will be sufficiently detailed that you can use them directly in planning. You can include them in a product backlog. Sometimes, however, to use stories in planning, you have to refine the stories to relate them more directly to the implementation of the system.

It is possible to express all of the functionality described in a scenario as user stories. So, an obvious question that you might ask is “Why not just develop user stories and forget about scenarios?” Some agile methods rely exclusively on user stories, but I think that scenarios are more natural and are helpful for the following reasons:

1. Scenarios read more naturally because they describe what a user of a system is actually doing with that system. People often find it easier to relate to this specific information rather than to the statement of wants or needs set out in a set of user stories.
2. When you are interviewing real users or checking a scenario with real users, they don't talk in the stylized way that is used in user stories. People relate better to the more natural narrative in scenarios.

3. Scenarios often provide more context—information about what users are trying to do and their normal ways of working. You can do this in user stories, but it means that they are no longer simple statements about the use of a system feature.

Scenarios and stories are helpful in both choosing and designing system features. However, you should think of scenarios and user stories as “tools for thinking” about a system rather than a system specification. Scenarios and stories that are used to stimulate thinking don’t have to be complete or consistent, and there are no rules about how many of each you need.

3.4 Feature identification

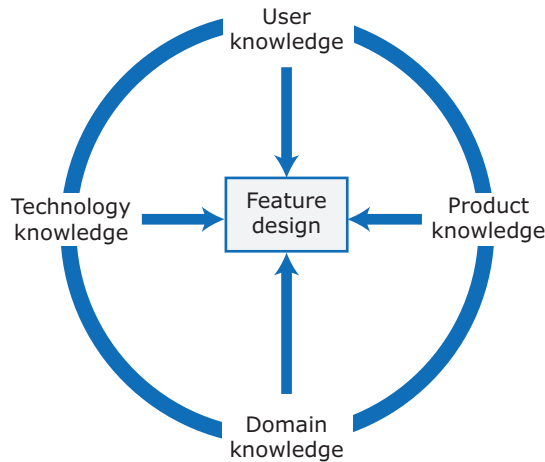
As I said in the chapter introduction, your aim at this early stage of product design is to create a list of features that define your software product. A feature is a way of allowing users to access and use your product’s functionality so that the feature list defines the overall functionality of the system. In this section, I explain how scenarios and stories can be used to help identify product features.

You should, ideally, identify product features that are independent, coherent and relevant:

1. *Independence* A feature should not depend on how other system features are implemented and should not be affected by the order of activation of other features.
2. *Coherence* Features should be linked to a single item of functionality. They should not do more than one thing, and they should never have side effects.
3. *Relevance* System features should reflect the way users normally carry out some task. They should not offer obscure functionality that is rarely required.

There is no definitive method for feature selection and design. Rather, the four important knowledge sources shown in Figure 3.8 can help with this.

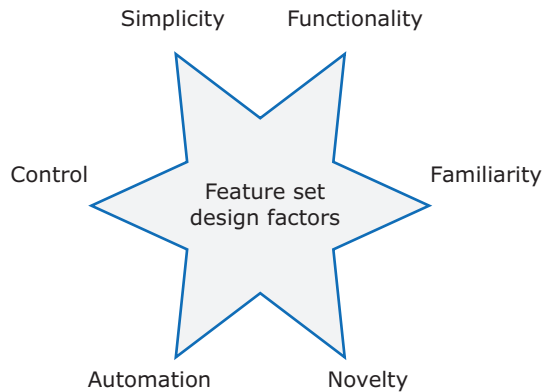
Table 3.8 explains these knowledge sources in more detail. Of course, these are not all of equal importance for all products. For example, domain knowledge is very important for business products but less important for generic consumer products. You therefore need to think carefully about the knowledge required for your specific product.

Figure 3.8 Feature design

Innovation often stems from a combination of domain and technology knowledge. A good example is the Git system for code management that I cover in Chapter 10. Git works in a completely different way from previous code management systems. These older systems were based on a technology model in which storage was expensive, so they focused on limiting storage use and delivering information to users as required. The developer of Git realized

Table 3.8 Knowledge required for feature design

Knowledge	Description
User knowledge	You can use user scenarios and user stories to inform the team of what users want and how they might use the software features.
Product knowledge	You may have experience of existing products or decide to research what these products do as part of your development process. Sometimes your features have to replicate existing features in these products because they provide fundamental functionality that is always required.
Domain knowledge	This is knowledge of the domain or work area (e.g., finance, event booking) that your product aims to support. By understanding the domain, you can think of new innovative ways of helping users do what they want to do.
Technology knowledge	New products often emerge to take advantage of technological developments since their competitors were launched. If you understand the latest technology, you can design features to make use of it.

Figure 3.9 Factors in feature set design

that storage had become much cheaper, so it was possible for all users to have a complete copy of all information. This allowed for a new approach that dramatically simplified software development by distributed teams.

When you are designing a product feature set and deciding how features should work, you have to consider the six factors shown in Figure 3.9.

Unfortunately, it is impossible to design a feature set in which all of these factors are optimized, so you have to make some trade-offs:

1. *Simplicity and functionality* Everyone says they want software to be as simple as possible to use. At the same time, they demand functionality that helps them do what they want to do. You need to find a balance between providing a simple, easy-to-use system and including enough functionality to attract users with a variety of needs.
2. *Familiarity and novelty* Users prefer that new software should support the familiar everyday tasks that are part of their work or life. However, if you simply replicate the features of a product that they already use, there is no real motivation for them to change. To encourage users to adopt your system, you need to include new features that will convince users that your product can do more than its competitors.
3. *Automation and control* You may decide that your product can automatically do things for users that other products can't. However, users inevitably do things differently from one another. Some may like automation, where the software does things for them. Others prefer to have control. You therefore have to think carefully about what can be automated, how it is automated, and how users can configure the automation so that the system can be tailored to their preferences.

Your choices have a major influence on the features to be included in your product, how they integrate, and the functionality they provide. You may make a specific choice—for example, to focus on simplicity—that will drive the design of your product.

One problem that product developers should be aware of and try to avoid is “feature creep.” Feature creep means that the number of features in a product creeps up as new potential uses of the product are envisaged.

The size and complexity of many large software products such as Microsoft Office and Adobe Photoshop are a consequence of feature creep. Most users use only a relatively small subset of the features of these products. Rather than stepping back and simplifying things, developers continually added new features to the software.

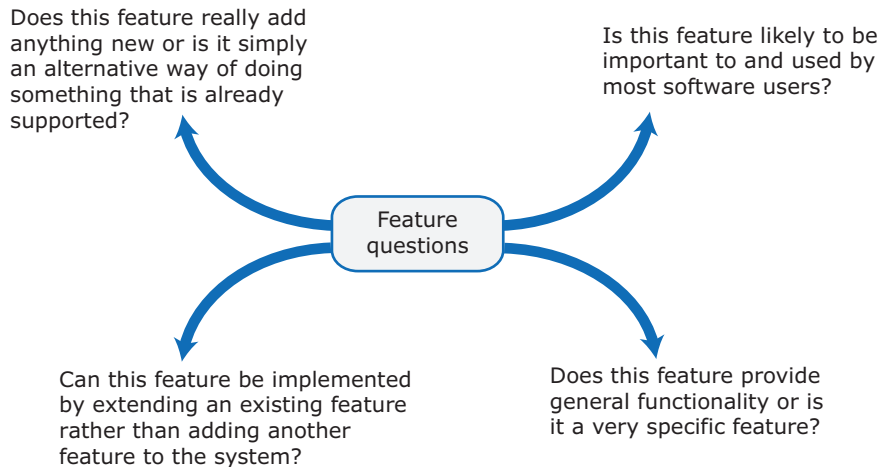
Feature creep adds to the complexity of a product, which means that you are likely to introduce bugs and security vulnerabilities into the software. It also usually makes the user interface more complex. A large feature set often means that you have to bundle vaguely related features together and provide access to these through a higher-level menu. This can be confusing, especially for less experienced users.

Feature creep happens for three reasons:

1. Product managers and marketing executives discuss the functionality they need with a range of different product users. Different users have slightly different needs or may do the same thing but in slightly different ways. There is a natural reluctance to say no to important users, so functionality to meet all of the users’ demands ends up in the product.
2. Competitive products are introduced with slightly different functionality to your product. There is marketing pressure to include comparable functionality so that market share is not lost to these competitors. This can lead to “feature wars,” where competing products become more and more bloated as they replicate the features of their competitors.
3. The product tries to support both experienced and inexperienced users. Easy ways of implementing common actions are added for inexperienced users and the more complex features to accomplish the same thing are retained because experienced users prefer to work that way.

To avoid feature creep, the product manager and the development team should review all feature proposals and compare new proposals to features that have already been accepted for implementation. The questions shown in Figure 3.10 may be used to help identify unnecessary features.

Figure 3.10 Avoiding feature creep



3.4.1 Feature derivation

When you start with a product vision or writing scenarios based on that vision, product features immediately come to mind. I discussed the iLearn system vision in Chapter 1 and I repeat it in Table 3.9.

I have highlighted phrases in this vision suggesting features that should be part of the product, including:

- a feature that allows users to access and use existing web-based resources;
- a feature that allows the system to exist in multiple different configurations;
- a feature that allows user configuration of the system to create a specific environment.

These features distinguish the iLearn system from existing VLEs and are the central features of the product.

Table 3.9 The iLearn system vision

FOR teachers and educators WHO need a way to <i>help students use web-based learning resources and applications</i>, THE iLearn system is an open learning environment THAT <i>allows the set of resources used by classes and students to be easily configured for these students and classes by teachers themselves</i>. UNLIKE Virtual Learning Environments, such as Moodle, the focus of iLearn is the learning process rather than the administration and management of materials, assessments, and coursework. OUR product <i>enables teachers to create subject and age-specific environments for their students</i> using any web-based resources, such as videos, simulations, and written materials that are appropriate

Table 3.10 Jack's scenario with highlighted phrases

Jack is a primary school teacher in Ullapool, teaching P6 pupils. He has decided that a class project should be focused around the fishing industry in the area, looking at the history, development, and economic impact of fishing.

As part of this, students are asked to gather and share reminiscences from relatives, use newspaper archives, and collect old photographs related to fishing and fishing communities in the area. *Students use an iLearn wiki to gather together fishing stories and SCRAN (a history archive) to access newspaper archives and photographs.* However, Jack also needs a photo-sharing site as he wants *pupils to take and comment on each others' photos and to upload scans of old photographs* that they may have in their families. He needs to be able to moderate posts with photos before they are shared, because pre-teen children can't understand copyright and privacy issues.

Jack *sends an email to a primary school teachers' group* to see if anyone can recommend an appropriate system. Two teachers reply and both suggest that he use KidsTakePics, a photo-sharing site that allows teachers to check and moderate content. As *KidsTakePics is not integrated with the iLearn authentication service*, he sets up a teacher and a class account with KidsTakePics.

He uses the the iLearn setup service to add KidsTakePics to the services seen by the students in his class so that when they log in, they can immediately use the system to upload photos from their phones and class computers.

This approach of highlighting phrases in a narrative description can be used when analyzing scenarios to find system features. You read through the scenarios, look for user actions (usually denoted by active verbs, such as “use,” “choose,” “send,” “update,” and so on), and highlight the phrases where these are mentioned. You then think about the product features that can support these actions and how they might be implemented.

In Table 3.10, I have done this with Jack's scenario (see Table 3.5), in which he sets up a system for his students' project work.

The highlighted text identifies features and feature characteristics that should be part of the iLearn system:

- a wiki for group writing;
- access to the SCRAN history archive, which is a shared national resource that provides access to historical newspaper and magazine articles for schools and universities;
- the ability to set up and access an email group;
- the ability to integrate some applications with the iLearn authentication service.

It also confirms the need for the configuration feature that has already been identified from the product vision.

Feature identification should be a team activity, and as features are identified, the team should discuss them and generate ideas about related features. Jack's scenario suggests that there is a need for groups to write together. You should therefore think about age-appropriate ways to design features for:

- collaborative writing, where several people can work simultaneously on the same document;
- blogs and web pages as a way of sharing information.

You can also think about generalizing the features suggested by the scenario. The scenario identifies the need for access to an external archive (SCRAN). However, perhaps the feature that you add to the software should support access to any external archive and allow students to transfer information to the iLearn system.

You can go through a similar process of highlighting phrases in all of the scenarios that you have developed and using them to identify and then generalize a set of product features. If you have developed user stories to refine your scenarios, these may immediately suggest a product feature or feature characteristic. For example, this story was derived from Emma's scenario (see Table 3.6):

As a teacher and a parent, I want to be able to select the appropriate iLearn account so that I don't have to have separate credentials for each account.

This story states that the account feature of the iLearn system has to accommodate the idea that a single user may have multiple accounts. Each account is associated with a particular role the user may adopt when using the system. When logging in, users should be able to select the account they wish to use.

3.4.2 The feature list

The output of the feature identification process should be a list of features that you use for designing and implementing your product. There is no need to go into a lot of detail about the features at this stage. You add detail when you are implementing the feature.

You may describe the features on the list using the input/action/output model that I showed in Figure 3.2. Alternatively, you can use a standard template, which includes a narrative description of the feature, constraints that have to be considered, and other relevant comments.

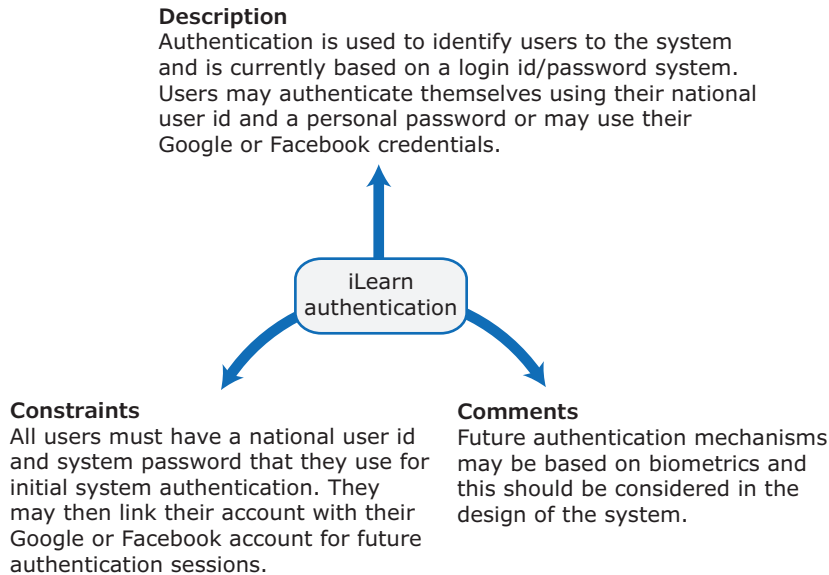
Figure 3.11 The iLearn authentication feature

Figure 3.11 is an example of this feature template that is used to describe the system authentication feature in the iLearn system.

The descriptions associated with the feature can sometimes be very simple. For example, a Print feature might be described using the simple feature template that I introduced at the beginning of the chapter:

Print the document to a selected printer or to PDF.

Alternatively, you can describe a feature from one or more user stories. Descriptions based on user stories are particularly useful if you intend to use Scrum and story-based planning when developing the software.

Table 3.11 shows how you can describe the configuration feature of the iLearn system using user stories and the feature template shown in Figure 3.11. In this example, I have used an alternative text-based form of the feature template. This is useful when you have relatively long feature descriptions. Notice that the table includes user stories from the system manager and a teacher.

The product development team should meet to discuss the scenarios and stories, and it makes sense to set out the initial list of features on a whiteboard. This can be done using a web-based discussion, but these are less effective than a face-to-face meeting. The feature list should then be recorded in a

Table 3.11 Feature description using user stories

iLearn system configuration
Description As a system manager, I want to create and configure an iLearn environment by adding and removing services to/from that environment so that I can create environments for specific purposes. As a system manager, I want to set up sub-environments that include a subset of services that are included in another environment. As a system manager, I want to assign administrators to created environments. As a system manager, I want to limit the rights of environment administrators so that they cannot accidentally or deliberately disrupt the operation of key services. As a teacher, I want to be able to add services that are not integrated with the iLearn authentication system.
Constraints The use of some tools may be limited for license reasons so there may be a need to access license management tools during configuration.
Comments Based on Elena's and Jack's scenarios

shared document such as a wiki, a Google Sheet, or an issue-tracking system such as JIRA. Feature descriptions may then be updated and shared as new information about the features emerges.

When you have developed an initial list of feature ideas, you should either extend your existing prototype or create a prototype system to demonstrate these features. As I said in Chapter 1, the aim of software prototyping is to test and clarify product ideas and to demonstrate your product to management, funders, and potential customers. You should focus on the novel and critical features of your system. You don't need to implement or demonstrate routine features such as Cut and Paste or Print.

When you have a prototype and have experimented with it, you will inevitably discover problems, omissions, and inconsistencies in your initial list of features. You then update and change this list before moving on to the development of your software product.

I think that scenarios and user stories should always be your starting point for identifying product features. However, the problem with basing product designs on user modeling and research is that it locks in existing ways of working. Scenarios tell you how users work at the moment; they don't show how they might change their ways of working if they had the right software to support them.

User research, on its own, rarely helps you innovate and invent new ways of working. Famously, Nokia, then the world leader in mobile (cell) phones, did

extensive user research and produced better and better conventional phones. Then Apple invented the smartphone without user research, and Nokia is now a minor player in the phone business.

As I said, stories and scenarios are tools for thinking; the most important benefit of using them is that you gain an understanding of how your software might be used. It makes sense to start by identifying a feature set from stories and scenarios. However, you should also think creatively about alternative or additional features that help users to work more efficiently or to do things differently.

KEY POINTS

- A software product feature is a fragment of functionality that implements something a user may need or want when using the product.
- The first stage of product development is to identify the list of product features in which you name each feature and give a brief description of its functionality.
- Personas are “imagined users”—character portraits of types of users you think might use your product.
- A persona description should paint a picture of a typical product user. It should describe the user’s educational background, technology experience, and why they might want to use your product.
- A scenario is a narrative that describes a situation where a user is accessing product features to do something that they want to do.
- Scenarios should always be written from the user’s perspective and should be based on identified personas or real users.
- User stories are finer-grain narratives that set out, in a structured way, something that a user wants from a software system.
- User stories may be used to extend and add detail to a scenario or as part of the description of system features.
- The key influences in feature identification and design are user research, domain knowledge, product knowledge, and technology knowledge.
- You can identify features from scenarios and stories by highlighting user actions in these narratives and thinking about the features that you need to support these actions.